Software Installation and Management

TEKsystems Global Services

WORKFORCE DEVELOPMENT

SOFTWARE INSTALLATION AND MANAGEMENT

SMP/E

System Modification Program Extended

The package manager for z/OS

z/OSMF

z/OS Management Facility

Browser-based front-end to z/OS system management tools; also exposes APIs used by external tools.

Ansible

An external tool that can interact with z/OSMF via ssh.

SOFTWARE INSTALLATION AND MANAGEMENT

SMP/E

System Modification Program Extended

The package manager for z/OS

z/OSMF

z/OS Management Facility

Browser-based front-end to z/OS system management tools; also exposes APIs used by external tools.

Ansible

An external tool that can interact with z/OSMF via ssh.

WHAT TYPE OF SOFTWARE IS SMP/E?

SMP/E is a *package manager*.

Installing software on a computer system comes down to placing certain files on peripheral devices where the computer system can find them, and preparing executables to be loaded and executed on the system.

It's possible to install software on any computer system by manually copying files and building executables. Any system that's used in a serious way will quickly become an unmanageable mess if people try to manage the system in that way, whether it's someone's personal device or a large-scale corporate system.

Quite early in the history of the software industry, people developed software that installs other software. It keeps track of dependencies between software products and manages the sequence of installation or removal of software products. They're called *package managers*.

You've worked with more than one package manager in your career so far, and probably in your personal life, too.

SMP/E is the one used for z/OS systems.

Other IBM operating systems that can run on the IBM Z have their own package managers.

WHAT'S A PACKAGE?

So, we're managing packages. Great. What's a *package*, then?

Some computer programs might be self-contained in a single file. They don't need any libraries or other products installed on the system. But that's the exception, not the norm.

Most software products comprise multiple files of various kinds – source code, object code, executables, documentation, configuration files, internal databases, GUI templates, localization values, examples to help users learn to work with the product, and more.

The collection of files necessary to make a software product usable is called a *package*. A package manager keeps track of versions, dependencies, and other details about packages.

The good news is IBM uses the same buzzword for this as the rest of the world: *package*. The bad news is there are more buzzwords coming soon.

Here's one now: *element*. The individual items contained in a package are called *elements*.

SYSTEM MODIFICATIONS

When you change any of the software on a computer system in any way (new install, upgrade, bug fix, uninstall), you're modifying the system.

Up until roughly 2000, IBM tended to name their products by the job they did. The original version of z/OS was called "Operating System." IMS stands for "Information Management System." DB2 means "Data Base," too.

They didn't name their products things like "Python" or "Homebrew" or "Ansible." You don't have to guess that System Modification Program is for modifying the system.

And the modifications to the system are called *system modifications*, or SYSMODs.

CATEGORIES OF SYSMODS

There are four types of SYSMODs.

- **Function SYSMOD** – install a new product or a new version or release of a product that's already present on the system.
- **PTF SYSMOD** – Program Temporary Fix or Product Temporary Fix (PTF) is an IBM-supplied correction for a known bug.
- **APAR SYSMOD** – Another kind of fix. IBM calls a bug report an APAR (Authorized Program Analysis Report). This type of SYSMOD fixes an APAR. APARs can be pretty minor, while PTFs are more significant.
- **USERMOD** – A "user" modification to a product (IBM or other), such as a user exit routine. These need to be tracked and managed because user modifications create differences between the base product and the product as customized in a particular installation. Subsequent SYSMODs for the base product shouldn't destroy the user's customization. Also, a user modification can introduce a dependency that the base product alone doesn't have. That has to be managed.

FUNCTION MODIFICATION ID (FMID)

SYSMODs are uniquely identified by a value called the Function Modification ID, or FMID (pronounced "eff-mid").

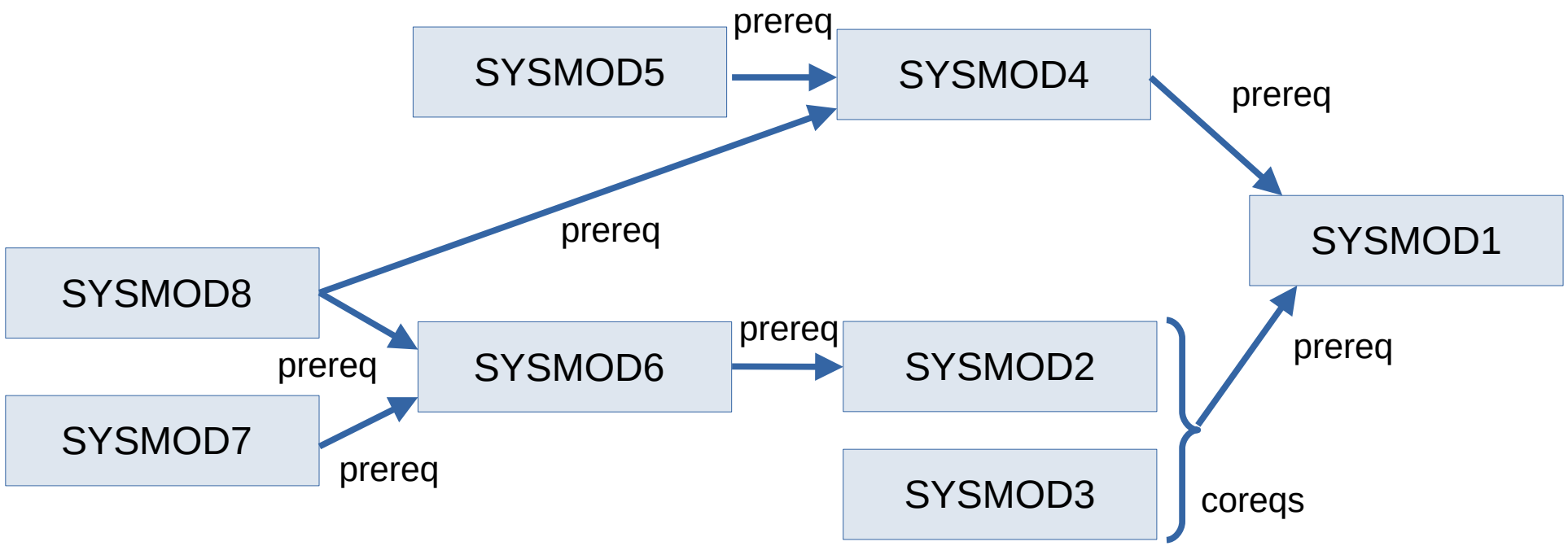
This information is dynamic and changes frequently. The z/OSMF facility, which front-ends SMP/E, has a screen that lets you query the FMIDs that are currently on your own system.

The illustration at right came from an obsolete list of FMIDs that IBM used to maintain as a public reference. They found it was tedious to keep the list up to date, and not that useful to customers after all. It gives you an idea of what FMIDs look like and how they relate to products.

FMID	NAME			
EAS1102	ASSEMBLER XF	5752SC103	R102	MVSS038J
EB81102	BASE CONTROL PROGRAM	5752SC***	R102	MVSS038J
EBT1102	BTAM	5752SC120	R102	MVSS038J
EDE1102	DEMF	5752CM100	R102	MVSS038J
EDM1102	DATA MANAGEMENT	5752SCID*	R102	MVSS038J
EDS1102	DASD ERP	5752SC1C*	R102	MVSS038J
EDU1602	ICKDSF R068 MVS/XA	565525701	R602	ICKDSF060
EDU1702	ICKDSF R070 MVS/XA	565525701	R702	ICKDSF070
EDU1802	ICKDSF R080 MVS/XA	565525701	R802	ICKDSF080
EEP0100	370X-EP PEP	5735SC100	R100	EP030
EEP0101	EP/OS	5735SC100	R101	EP030
EEP0200	EP/OS WITH ACF/NCP R2	5735SC100	R200	EP030
EEP0300	EP FOR NCP REL3	5735SC100	R300	EP030

REQUISITE CHAINS

PREREQ (prerequisite) must be installed before a given SYSMOD
COREQs (corequisites) must be installed together



The illustration suggests our goal is to install SYSMOD1. To do so, we must ensure the prerequisite SYSMODs are installed in the correct order, and corequisites are installed together. The whole sequence is called the *requisite chain* for SYSMOD1.

SMP/E MODIFICATION CONTROL STATEMENTS

SMP/E runs as a batch program that is controlled by control statements called *Modification Control Statements* (MCS). The MCS function is a declarative language we use to direct SMP/E to perform its various functions.

Define an existing PDS as a TLIB.

```
//SMPCNTL DD *
 SET BDY(TSTTGT).

 UCLIN.
   ADD DDDEF(TGTLOAD)
       DA('HLQ.TSTTGT.LOAD').
 ENDUCL.
/*
```

Receive SYSMODs.

```
//SMPCNTL DD *
 SET BDY(GLOBAL).
 RECEIVE SYSMODS.
/*
```

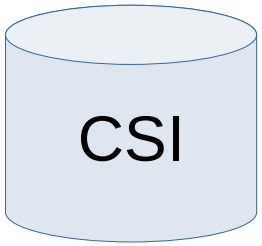
See what would happen if changes were applied.

```
//SMPCNTL DD *
 SET BDY(TSTTGT).
 APPLY CHECK
   S(UM00001).
/*
```

Apply the changes.

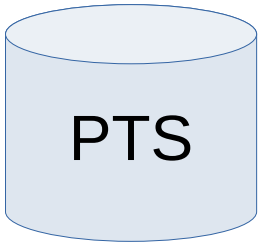
```
//SMPCNTL DD *
 SET BDY(TSTTGT).
 ACCEPT CHECK
   S(UM00001).
/*
```

SMP/E DATA SETS



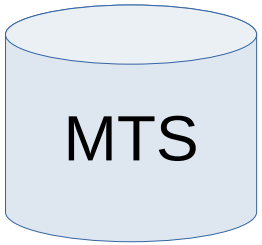
Consolidated Software Inventory

SMP/E uses the CSI to track the distribution and target libraries.



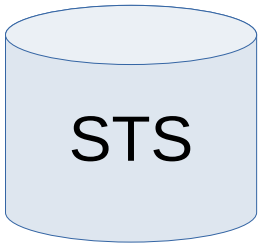
PTF Temporary Storage

This data set holds PTF SYSMODs as received.



Macro Temporary Storage

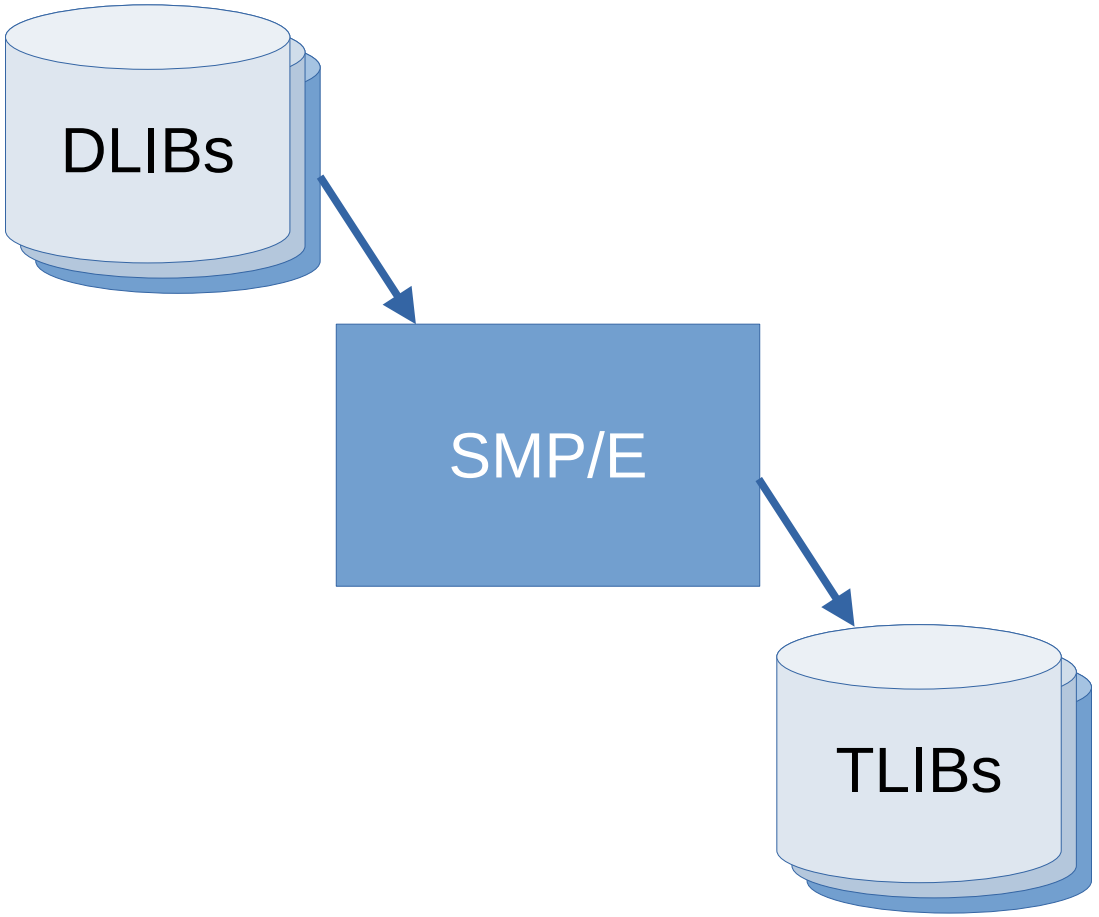
This data set holds macros used in assemblies during APPLY processing.



Source Temporary Storage

This data set holds source code used in assemblies during APPLY processing.

DLIBS AND TLIBS

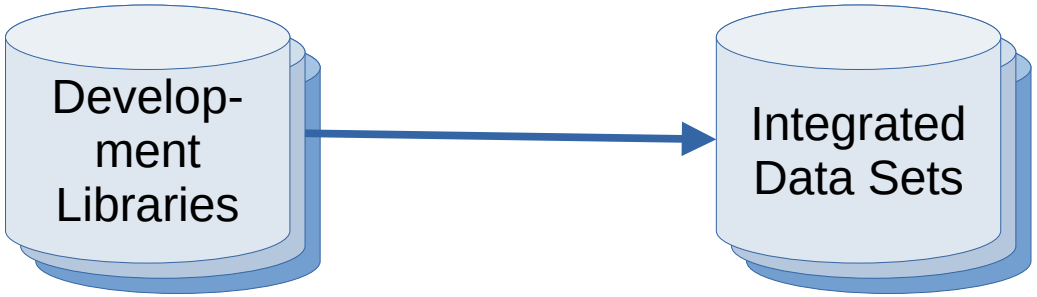


SMP/E works with two types of libraries: *Distribution Libraries* (TLIBs) and *Target Libraries* (TLIBs). They aren't physically different types of artifacts; the distinction is in how they're used.

A DLIB contains the master copy of the elements constituting each product on a system.

A TLIB is a target of the product installation process. It contains executables and other types of files that are needed to operate the product on the system.

PACKAGING



Step 1: Integrate the code.

- Create any required JCLIN data.
- Collect the JCLIN data and elements.
- Compile or assemble any macros and source.
- Bind or link-edit the resulting object modules.

Building a SYSMOD is called *packaging*.

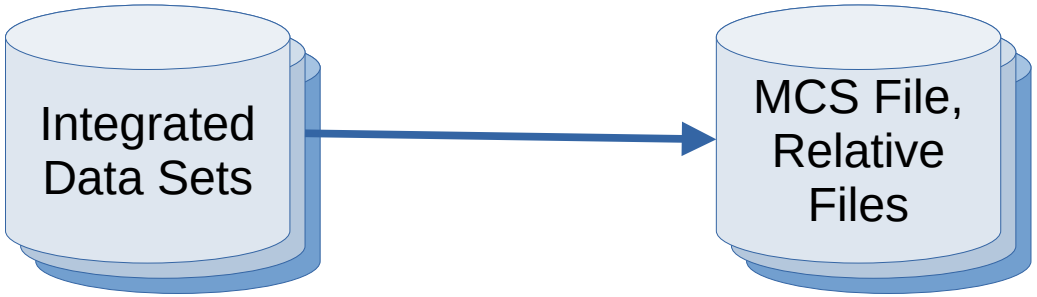
If you're involved with preparing a product release for installation with SMP/E, you'll be engaged in this process.

If you will be installing products with SMP/E but not preparing products for distribution, you'll be consuming the results of this process.

We'll step through the process for the sake of general understanding. The walkthrough assumes you're preparing the SYSMOD on an existing z/OS system, not Linux or Windows or XYZ.

A slide deck is included with the training material entitled *References*. Under *SMP/E: IBM Reference Docs*, the IBM document *Standard Packaging Rules for z/OS-Based Products* is listed. Consult that document for detailed information about packaging.

PACKAGING



Step 2: Build the SYSMOD.

- Create unloaded PDSs and other files needed for the installation.
- Create an MCS File containing MCS statements for SMP/E to process when working with the SYSMOD.
- Write the files to a RELFILE tape.
- Prepare a *Program Directory* – essentially an installation guide for the product.

Years ago, the way to transfer files between mainframes was via magnetic tape. The terminology remains the same to this day. You will probably transfer the SYSMOD to destination z/OS instances by other means besides tape.

An unloaded PDS is a library that has been converted into a single sequential data set by the IEBCOPY utility. An unloaded PDS can't be used directly as a library, but it can be reconstituted by IEBCOPY on the destination z/OS instance.

The other files needed in the SYSMOD will be either text files or binaries produced by compilers and assemblers. They may be executables produced by a z/OS binder or linkage editor.

The MCS file contains control statements for SMP/E to process the relative files.

DEFINING THE REQUISITE CHAIN

Most package managers will resolve all the prerequisites for the package you want to install. You don't have to know what they are in advance.

SMP/E is a little more limited in that area. It can look up prerequisites in the CSI data set on the z/OS instance, but it doesn't automatically identify the entire requisite chain for the SYSMOD you're applying. You usually need to specify the complete requisite chain in the SYSMOD.

In reality, it isn't that bad. In most cases, your organization will have installed the prerequisites for anything new you buy from IBM or its business partners. If you can specify the prerequisites that you know about, the prerequisites for *those* prerequisites will probably already be stored in the CSI. If you have issues with the installation due to missing prereqs, you can resolve them during the installation process.

DISTRIBUTING SOURCE

As you read the IBM documentation for SMP/E, you'll notice there's an unstated assumption that the SYSMODs were prepared on another, existing z/OS system.

Many are object code only (OCO) distributions, which means their sources were assembled or compiled into object modules, and the object modules were bound or link-edited into executables or load modules, before the SYSMOD was published.

That will be the case for components of z/OS itself, and many of the utilities that ship with it.

That can only happen on another z/OS system. There's no way to create z/OS executables on other operating systems.

You can also distribute a product in source form and specify that the code has to be compiled or assembled as part of the SMP/E installation process.

BASIC INSTALLATION STEPS

Although the underlying details are complicated, the steps to follow when installing a SYSMOD are straightforward.

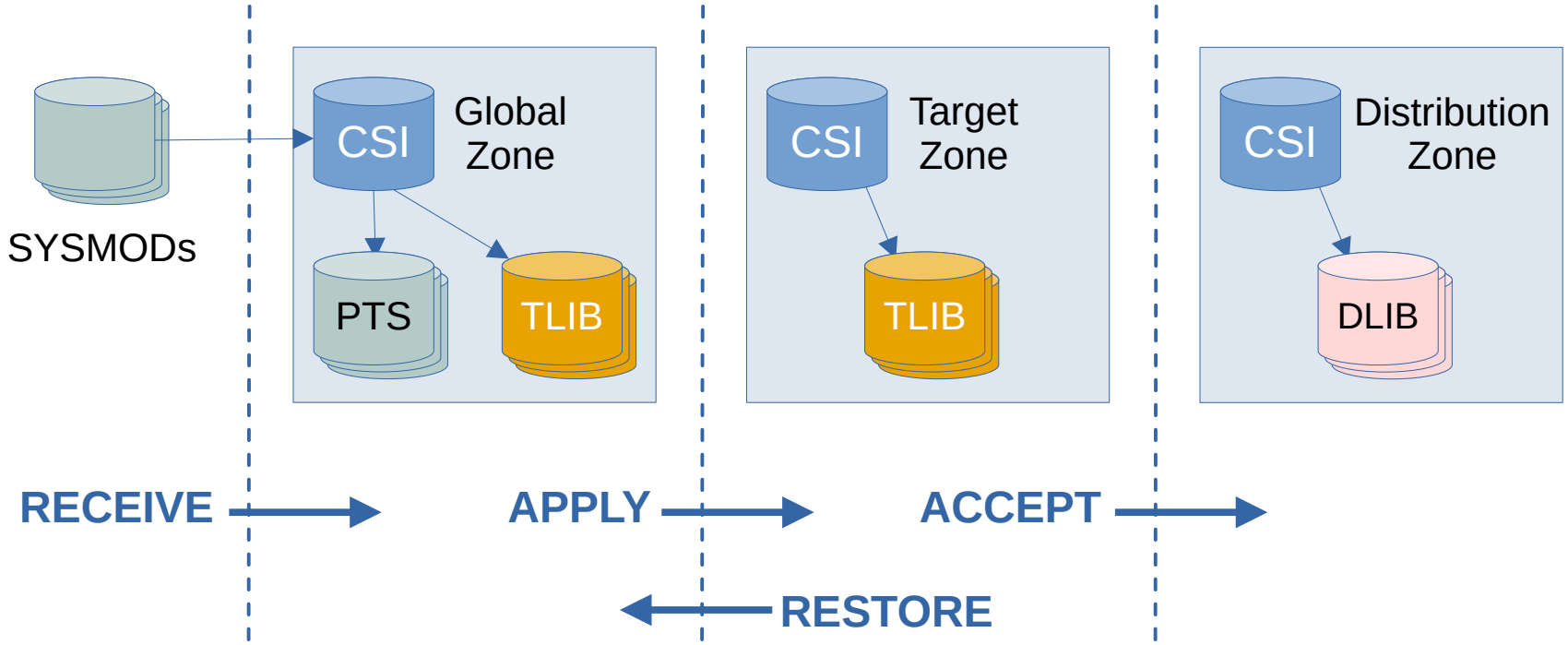


Illustration loosely based on IBM documentation: https://www.ibm.com/docs/en/zos-basic-skills?topic=smpe-process-installing-zos-elements-service

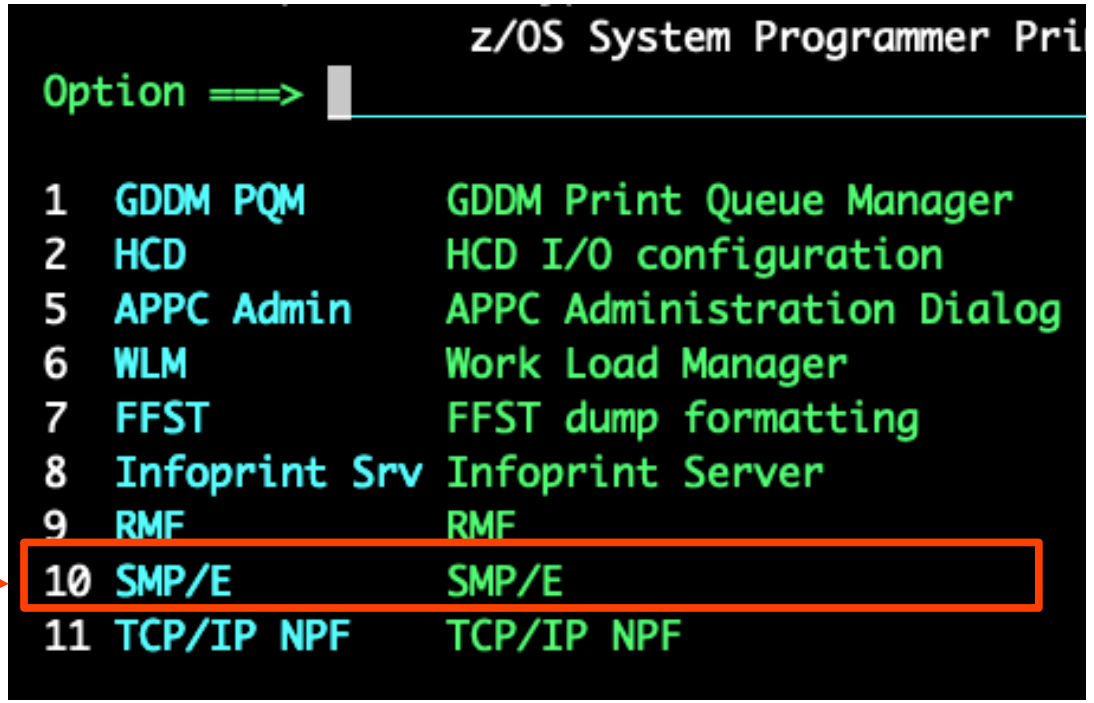
WHAT'S AVAILABLE?

IBM doesn't publish a general list of available SYSMODS, so you can't browse through them to see what's available.

When your organization buys or subscribes to a z/OS-based product, you'll receive the SYSMODs for whatever was purchased or licensed. After you RECEIVE them into SMP/E, you'll be able to browse what's on your system.

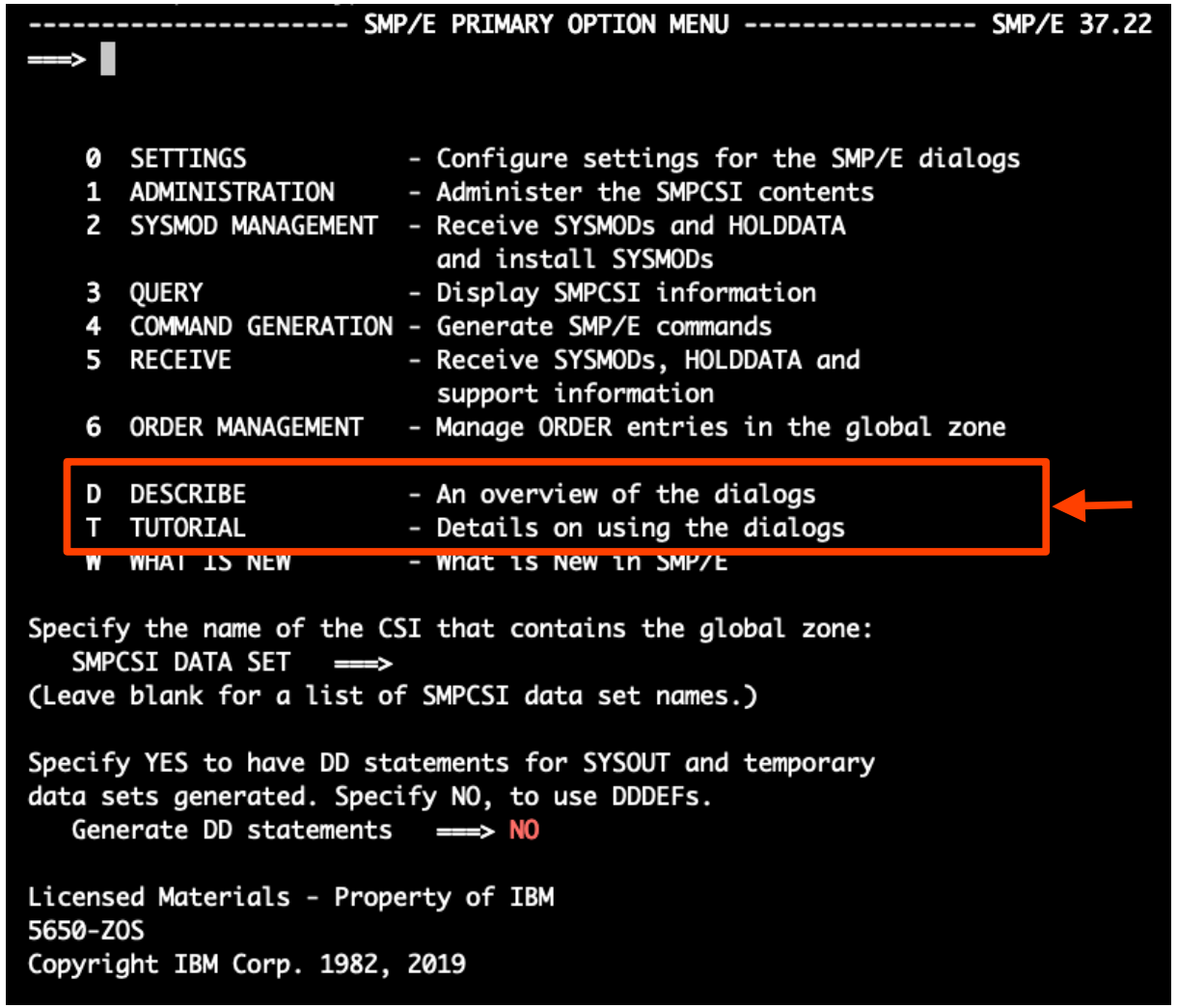
Let's see what's on the lab system right now.

Your "real" ISPF menus will look different from the lab setup. Don't panic. If you're authorized, a menu option to access SMP/E will be available somewhere in your ISPF menu hierarchy.



Our lab system is set up with a "z/OS System Programmer Primary Option Menu" under ISPF. You can find SMP/E there.

INITIAL SMP/E ISPF DISPLAY



The ISPF panels offer a good way to begin to get familiar with SMP/E. In the upper right-hand corner, you see the version of SMP/E that's installed on this z/OS instance.

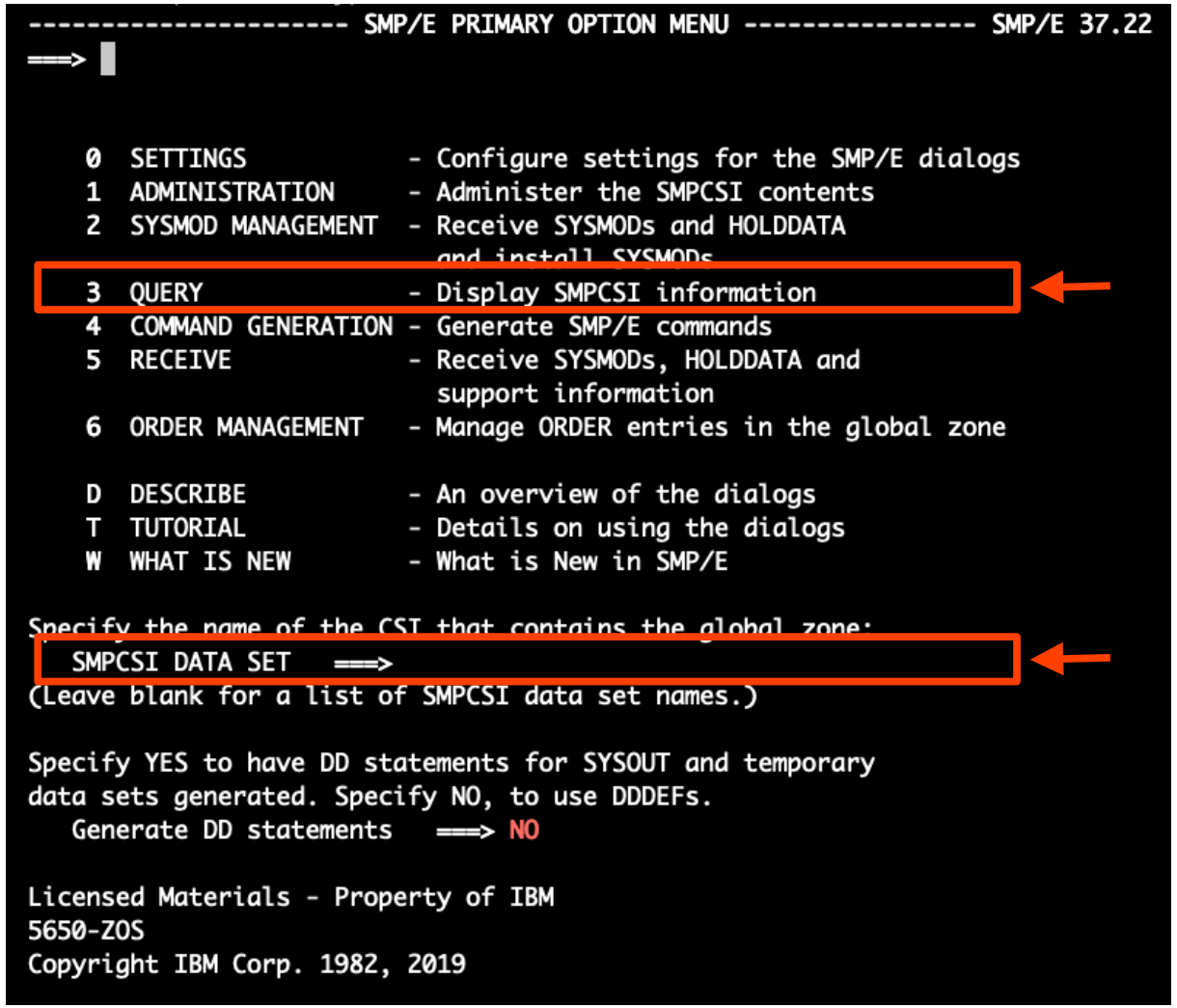
When you're looking for documentation, online resources, or help from humans or LLMs, you need to know the SMP/E version. Commands and behavior vary between versions, sometimes significantly.

You can choose from among several options. Two of them will be useful to get acquainted with the facility: D for Describe and T for Tutorial.

We'll leave it to you to check those out during the next lab exercise.

For now, let's try a couple of the options and see what they do.

FINDING CSI DATA SETS

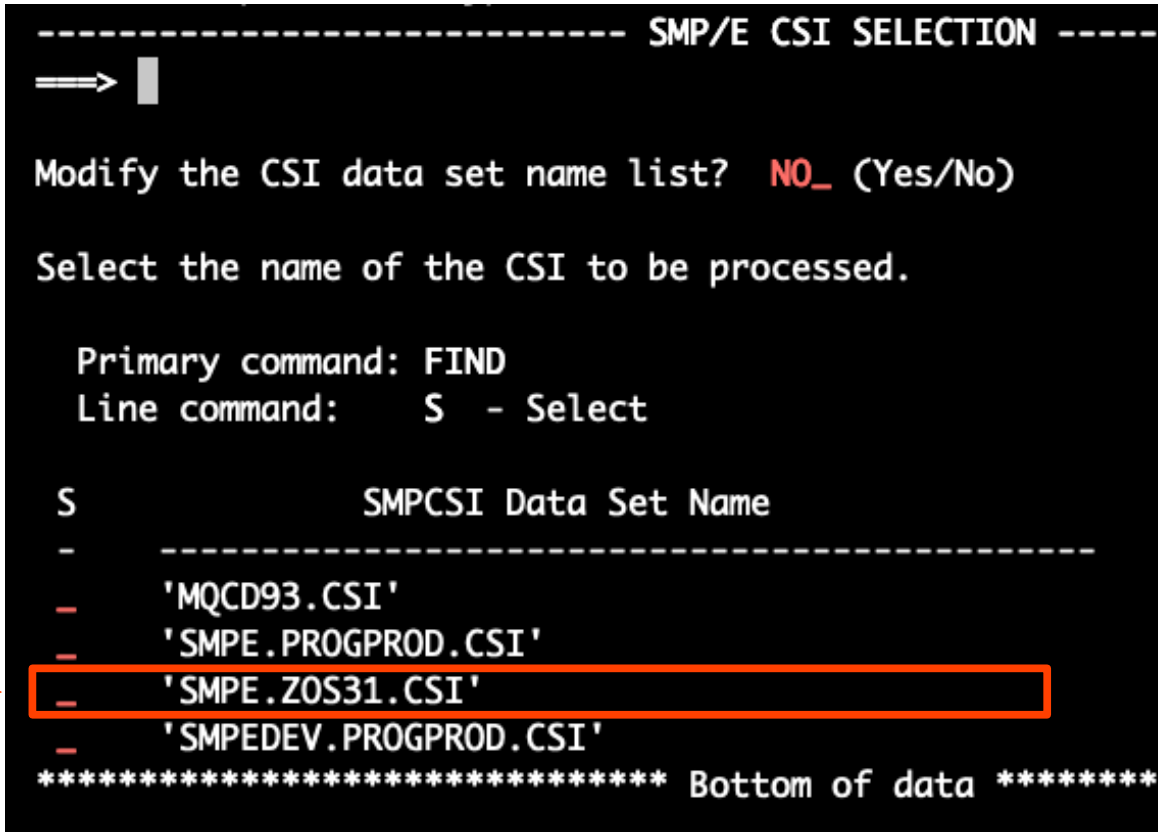


If you aren't sure where to start or you don't know the names of CSI data sets, you can choose option 3, Query, and leave the name of the SMPCSI data set blank.

FINDING CSI DATA SETS

That gets you here, to a list of the CSI data sets SMP/E knows about on this z/OS instance.

SMPE.ZOS31.CSI looks promising. The name suggests it's fundamental to the system.



QUERYING CSI DATA

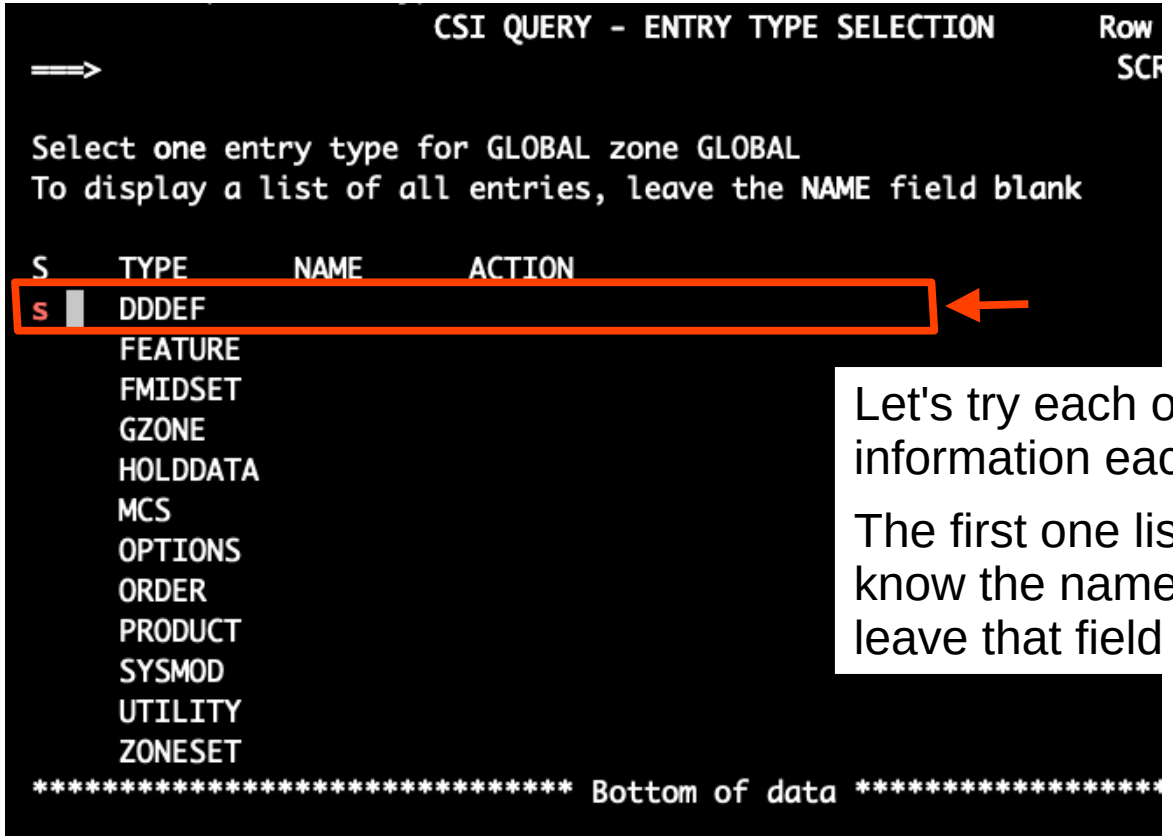
Next, we're presented with a few options to perform queries on the CSI data. Let's try option 1, CSI Query.

```
                        QUERY SELECTION MENU
===> 1

   1  CSI QUERY            - Display SMPCSI entries
   2  CROSS-ZONE QUERY     - Display status of an entry in
                             all zones
   3  SOURCEID QUERY       - Display SOURCEIDs for specified zone

   D  DESCRIBE             - Overview of using QUERY

   T  TUTORIAL             - Information on using QUERY

To return to the SMP/E primary option menu, enter  END .
```

```
                              CSI QUERY
===>

Specify the zone, entry type, and name to be queried:

   ZONE NAME    ===> global   Name of the zone to be queried.
                              To display a list of all zones,
                              leave blank

   ENTRY TYPE   ===>          Entry type to be queried.
                              To display a list of all valid
                              entry types, leave ENTRY TYPE
                              and ENTRY NAME blank

   ENTRY NAME   ===>          Entry name to be queried.
                              Leave blank or use a wildcard
                              (entry name pattern) to display
                              a selection list.

To return to the Query selection menu, enter END .
```

We don't know enough yet to specify exactly what we're looking for. Let's enter "global" zone and leave the other options blank.

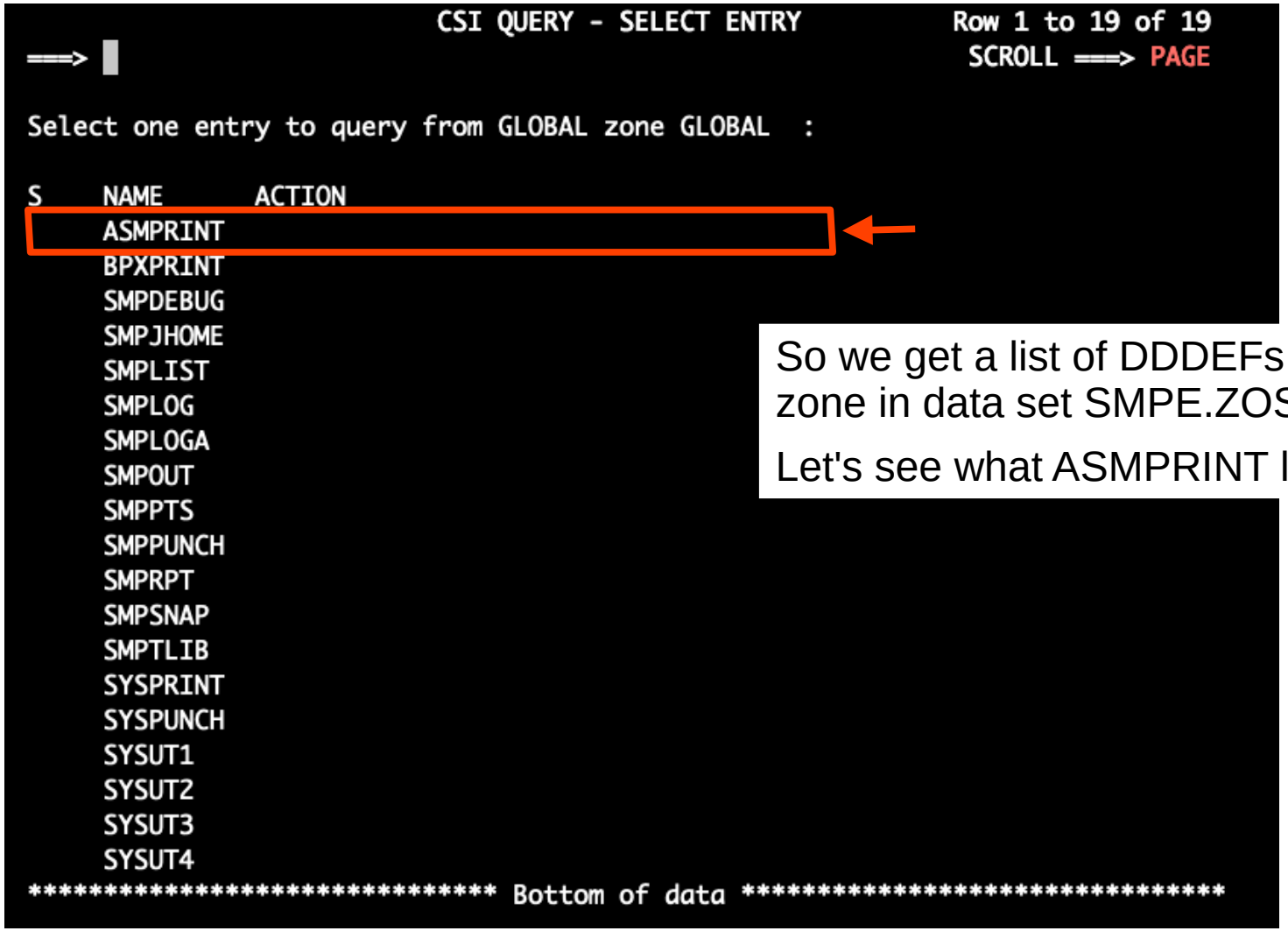
QUERYING DDDEF DATA



Let's try each option and see what kinds of information each one provides.

The first one listed is DDDEF. We don't know the names of any of them, so we'll leave that field blank and see what we get.

QUERYING DDDEF DATA



So we get a list of DDDEFs from the GLOBAL zone in data set SMPE.ZOS31.CSI.

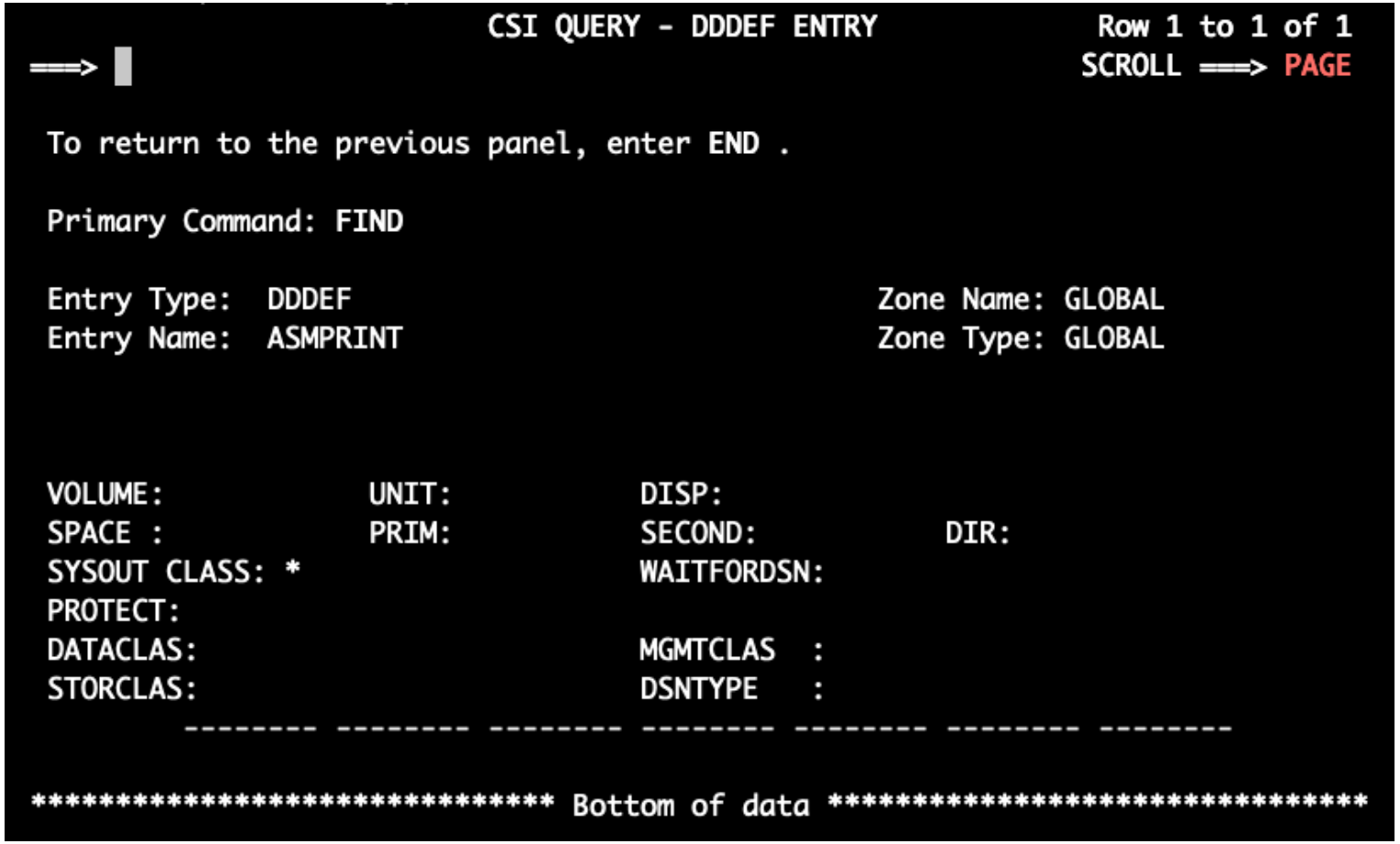
Let's see what ASMPRINT look like.

QUERYING DDDEF DATA

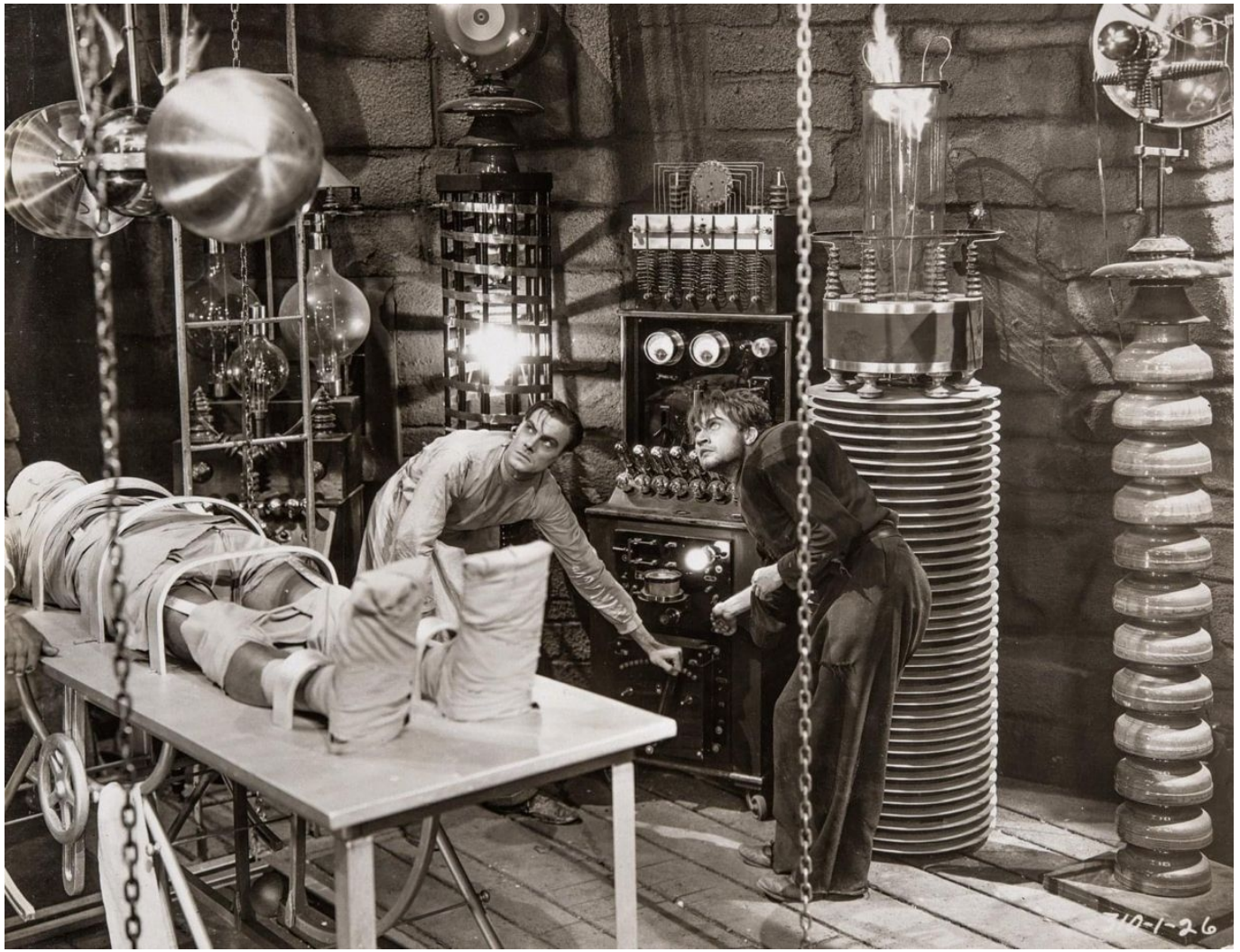
There's not much to see here. It might be here as a placeholder or example.

We're just exploring, to learn what we can about SMP/E. Even this empty form tells us something – the kinds of information a DDDEF stores.

But you don't need an instructor to walk you through a process of exploration. You can do that in the next lab.



LAB Z/OS 37: EXPLORE SMP/E ISPF MENUS



A CLOSER LOOK AT THE CSI

A Consolidated Software Inventory (CSI) data set controls SMP/E processing and tracks the status and results of installation activities.

The information is divided into three categories called *zones*:

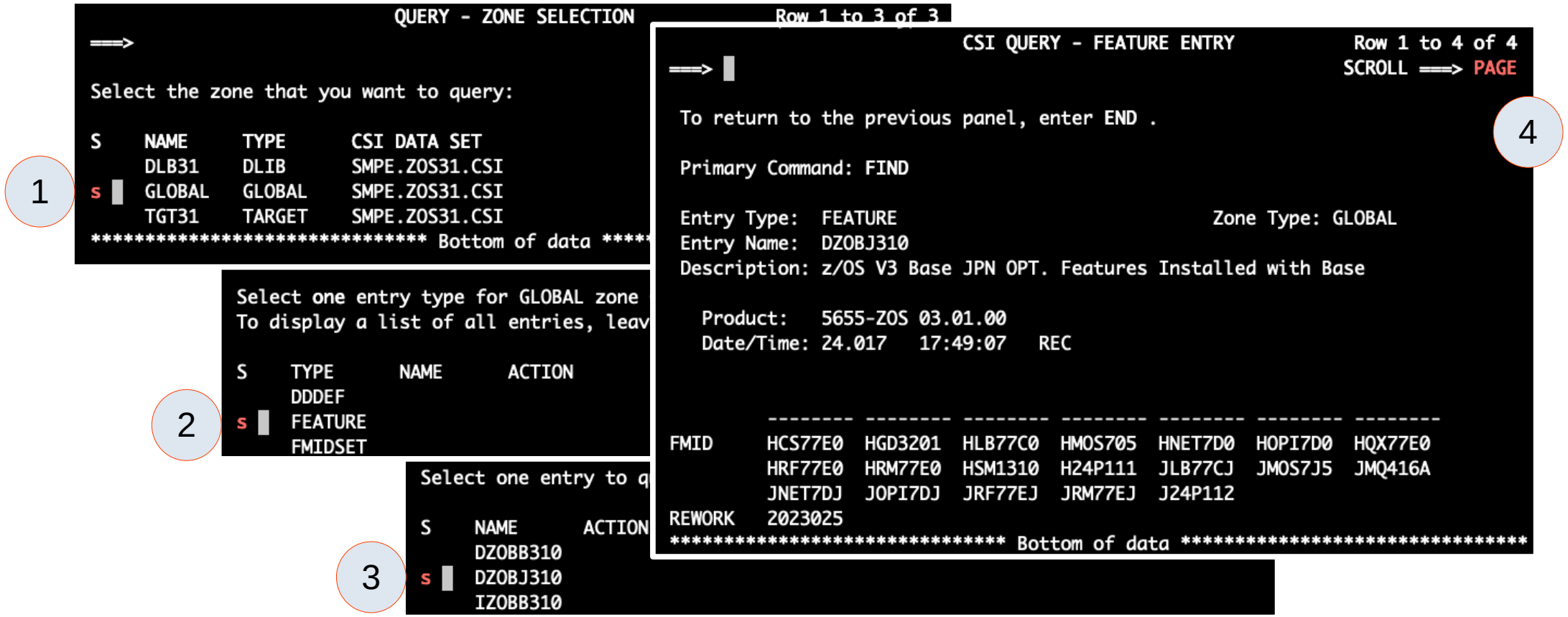
Global Zone – there's exactly one per CSI. This holds information about the SYSMODs.

DLIB Zone – there's at least one per CSI. This zone tracks the Distribution Libraries (used when building the artifacts to be installed).

Target Zone – there's at least one per CSI. This zone tracks the target libraries (where the installed product lives).

Global zone shown near upper right-hand corner of photo.

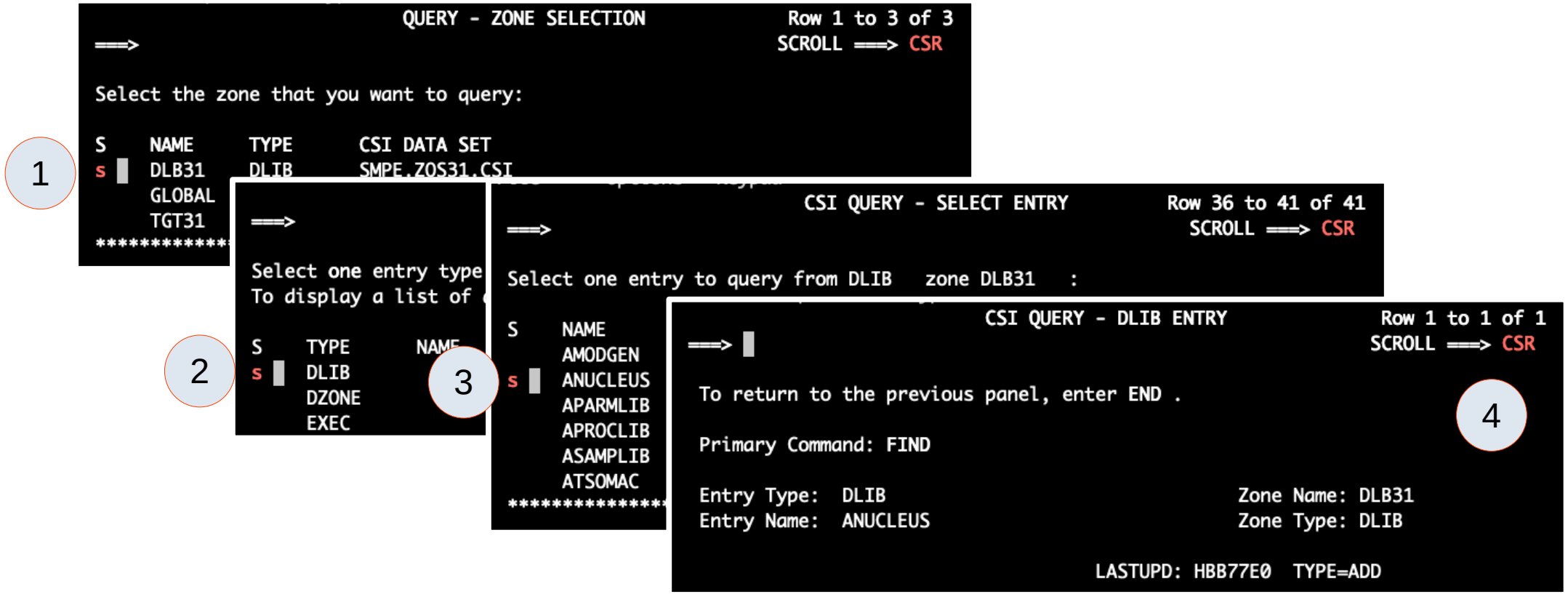
A CLOSER LOOK AT THE CSI – GLOBAL ZONE



Here you can see three zones exist in SMPE.ZOS31.CSI. Within each zone, information about one or more entry types is maintained. Zero or more entries are stored under each entry type.

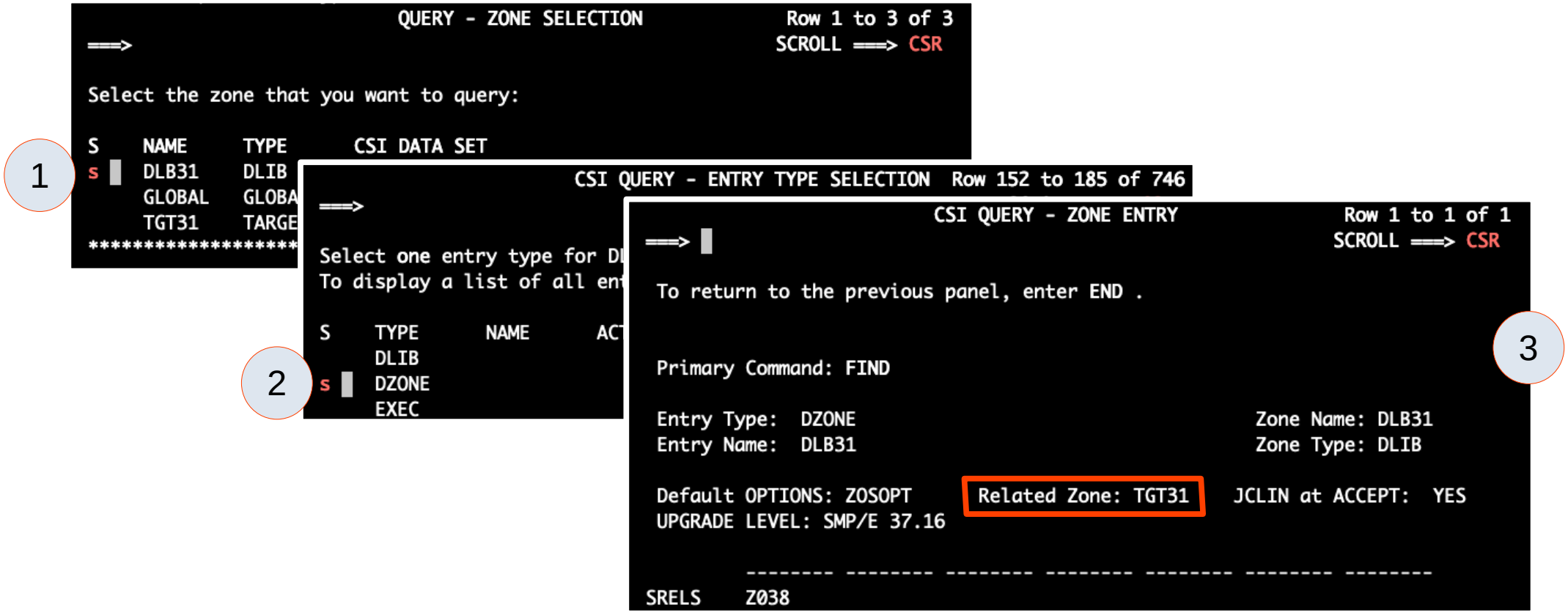
When querying in ISPF, you first choose the CSI data set, then the zone within that data set, then the entry type within that zone, and finally the entry you want to examine.

A CLOSER LOOK AT THE CSI – DLIB ZONE



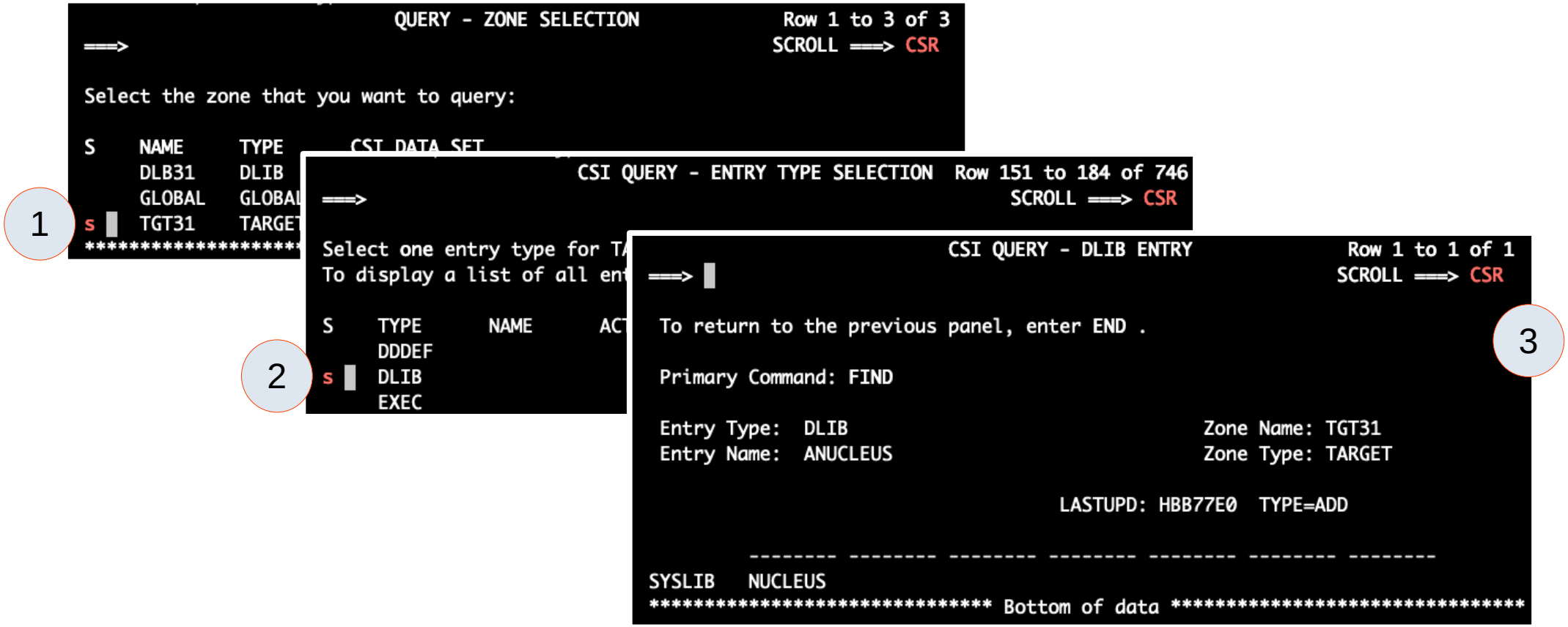
Now let's back up to the Zone Selection panel and choose the DLIB Zone (Distribution Zone). Then we choose Entry Type DLIB, and we'll look at the ANUCLEUS entry. This is obviously not the actual Nucleus code. The CSI contains labels or names of things that cross-reference entries in other Zones. SMP/E uses this information to guide its processing.

A CLOSER LOOK AT THE CSI – DLIB ZONE



Still in the DLIB Zone, we select Entry Type DZONE and look at entry DLB31. Like the previous example, this doesn't show detailed information about the distribution elements. We can see a reference to a Target Zone, TGT31. That's the name of a Target Zone in the same CSI (you can see it listed on the Zone Selection panel screenshot above). There can be more than one DLIB Zone and Target Zone in a CSI, so SMP/E needs to know which one goes with this entry.

A CLOSER LOOK AT THE CSI – TARGET ZONE



Looking at Target Zone TGT31 in the same CSI, we choose Entry Type DLIB and entry ANUCLEUS to see what kind of information the CSI contains in the Target Zone.

DATA SETS ASSOCIATED WITH SMPE.ZOS31

```
                          Data Set List Utility
Option ===>

   blank Display data set list              P Print data set list
       V Display VTOC information          PV Print VTOC information

Enter one or both of the parameters below:
   Dsname Level . . . SMPE.ZOS31
   Volume serial  . .
```

```
DSLIST - Data Sets Matching SMPE.ZOS31                    Row 1 of 11
Command ===>                                          Scroll ===> PAGE

Command - Enter "/" to select action                Message          Volume
-------------------------------------------------------------------------------
         SMPE.ZOS31.CSI                                              *VSAM*
         SMPE.ZOS31.HOLDDATA                                         T31VS1
         SMPE.ZOS31.IDX.INDEX                                        T31VS1
         SMPE.ZOS31.KSD.DATA                                         T31VS1
         SMPE.ZOS31.SMPLOG                                           T31VS1
         SMPE.ZOS31.SMPLOGA                                          T31VS1
         SMPE.ZOS31.SMPLTS                                           T31VS1
         SMPE.ZOS31.SMPMTS                                           T31VS1
         SMPE.ZOS31.SMPPTS                                           D31VS1
         SMPE.ZOS31.SMPSCDS                                          T31VS1
         SMPE.ZOS31.SMPSTS                                           T31VS1
****************************** End of Data Set list ****************************
```

SMP/E uses several other data sets besides the CSI to manage system modifications. Here we can see all the data sets that existed on the lab system as of the time the screenshots were taken whose HLQ and MLQ match those of the CSI data set, SMPE.ZOS31.CSI.

SMP/E creates these data sets when SYSMODs are received and processed, but all of them may not be used in every case.

DATA SETS ASSOCIATED WITH SMPE.ZOS31

```
                        Data Set List Utility
Option ===>

   blank Display data set list              P Print data set list
       V Display VTOC information          PV Print VTOC information

Enter one or both of the parameters below:
   Dsname Level . . . SMPE.ZOS31
   Volume serial  . .
```

```
DSLIST - Data Sets Matching SMPE.ZOS31                  Row 1 of 11
Command ===>                                          Scroll ===> PAGE

Command - Enter "/" to select action                Message          Volume
-------------------------------------------------------------------------------
        SMPE.ZOS31.CSI                                               *VSAM*
        SMPE.ZOS31.HOLDDATA                                          T31VS1
        SMPE.ZOS31.IDX.INDEX                                         T31VS1
        SMPE.ZOS31.KSD.DATA                                          T31VS1
        SMPE.ZOS31.SMPLOG                                            T31VS1
        SMPE.ZOS31.SMPLOGA                                           T31VS1
        SMPE.ZOS31.SMPLTS                                            T31VS1
        SMPE.ZOS31.SMPMTS                                            T31VS1
        SMPE.ZOS31.SMPPTS                                            D31VS1
        SMPE.ZOS31.SMPSCDS                                           T31VS1
        SMPE.ZOS31.SMPSTS                                            T31VS1
****************************** End of Data Set list ****************************
```

When we say a SYSMOD has been *received*, we don't mean that it has been stored in a data set on our system. We mean the SMP/E RECEIVE process has been done.

At that time, SMP/E creates CSI records for it, creates the SMPPTS data set, and stores the SYSMOD in that data set in the form of *MCS Members*.

The data set has a special format that is only used by SMP/E. You can browse its contents, but it will look "funny." There may be sections of plain text and sections of object code.

It is a kind of "library" in the general sense, but it is not a PDS or PDS/E.

DATA SETS ASSOCIATED WITH SMPE.ZOS31

```
                          Data Set List Utility
Option ===>

   blank Display data set list              P Print data set list
       V Display VTOC information          PV Print VTOC information

Enter one or both of the parameters below:
   Dsname Level . . . SMPE.ZOS31
   Volume serial  . .
```

```
DSLIST - Data Sets Matching SMPE.ZOS31                    Row 1 of 11
Command ===>                                          Scroll ===> PAGE

Command - Enter "/" to select action                Message          Volume
-------------------------------------------------------------------------------
         SMPE.ZOS31.CSI                                                *VSAM*
         SMPE.ZOS31.HOLDDATA                                           T31VS1
         SMPE.ZOS31.IDX.INDEX                                          T31VS1
         SMPE.ZOS31.KSD.DATA                                           T31VS1
         SMPE.ZOS31.SMPLOG                                             T31VS1
         SMPE.ZOS31.SMPLOGA                                            T31VS1
         SMPE.ZOS31.SMPLTS                                             T31VS1
         SMPE.ZOS31.SMPMTS                                             T31VS1
         SMPE.ZOS31.SMPPTS                                             D31VS1
         SMPE.ZOS31.SMPSCDS                                            T31VS1
         SMPE.ZOS31.SMPSTS                                             T31VS1
****************************** End of Data Set list ***********************
```

We mentioned that many SYSMODS are OCO – object code only – but it's also possible to install and build source. In those cases, data set SMPMTS contains macros used in assembling the source code, and SMPSTS contains source code .

DATA SETS ASSOCIATED WITH SMPE.ZOS31

```
                          Data Set List Utility
Option ===>

   blank Display data set list              P Print data set list
       V Display VTOC information          PV Print VTOC information

Enter one or both of the parameters below:
   Dsname Level . . . SMPE.ZOS31
   Volume serial  . .
```

```
DSLIST - Data Sets Matching SMPE.ZOS31                    Row 1 of 11
Command ===>                                          Scroll ===> PAGE

Command - Enter "/" to select action                Message          Volume
------------------------------------------------------------------------------
        SMPE.ZOS31.CSI                                               *VSAM*
        SMPE.ZOS31.HOLDDATA                                          T31VS1
        SMPE.ZOS31.IDX.INDEX                                         T31VS1
        SMPE.ZOS31.KSD.DATA                                          T31VS1
        SMPE.ZOS31.SMPLOG                                            T31VS1
        SMPE.ZOS31.SMPLOGA                                           T31VS1
        SMPE.ZOS31.SMPLTS                                            T31VS1
        SMPE.ZOS31.SMPMTS                                            T31VS1
        SMPE.ZOS31.SMPPTS                                            D31VS1
        SMPE.ZOS31.SMPSCDS                                           T31VS1
        SMPE.ZOS31.SMPSTS                                            T31VS1
****************************** End of Data Set list ****************************
```

You can tell SMP/E to hold a SYSMOD if it's not a good idea to install it immediately, or if there were problems in applying it and you need to fix a few things before re-trying it.

When that happens, SMP/E saves the Reason Report from APPLY processing in the HOLDDATA data set.

HOLDDATA EXAMPLE

```
BROWSE    SMPE.ZOS31.HOLDDATA(TGT31A) - 01.00      Line 0000000000 Col 001 080
Command ===>                                                 Scroll ===> CSR
******************************** Top of Data ********************************
BYPASSED HOLD REASON REPORT FOR APPLY PROCESSING

TYPE    REASON ID  FMID     SYSMOD   ++HOLD DATA
------  ---------  -------  -------  ----------------------------------------
SYSTEM  ACTION     HBB77E0  UJ93618  ++HOLD(UJ93618) SYSTEM FMID(HBB77E0) REASO
                                      COMMENT(
```

```
++HOLD DATA
------------------------------------------------------------------------
++HOLD(UJ93618) SYSTEM FMID(HBB77E0) REASON(ACTION) DATE(23205)
  COMMENT(
  ********************************************************************
  * FUNCTION AFFECTED: BCP   SMF                          (OA65166) *
  ********************************************************************
  * DESCRIPTION      : Recompile/relink application                  *
  ********************************************************************
  * TIMING           : Post-Apply                                    *
  ********************************************************************
  With this PTF, the IFAQUERY executable macro code is updated.
  Programs that call IFAQUERY must be recompiled to include the
  updated macro code provided by this PTF.  A program that is
  compiled with the updated macro code can run compatibly on a
  system that does not nave the PTF for OA65166 applied.
  ).

++HOLD(UJ93702) SYSTEM FMID(HBB77E0) REASON(ACTION) DATE(23220)
  COMMENT(
  The Stand-Alone Dump IPL volume must be rebuilt.
  ).
```

Here's a snippet from a member in a HOLDDATA data set. There's a lot more content that looks like this.

The lower screenshot is the same member after scrolling to the right so we can see all the text of the report.

The report cites a predefined reason category, in this case "action," identifies the system components affected, and provides an explanation of why the SYSMOD is held.

For reason "action," the report describes the actions necessary before the APPLY can be completed, or after the APPLY but still during the installation process.

RUNNING SMP/E IN BATCH

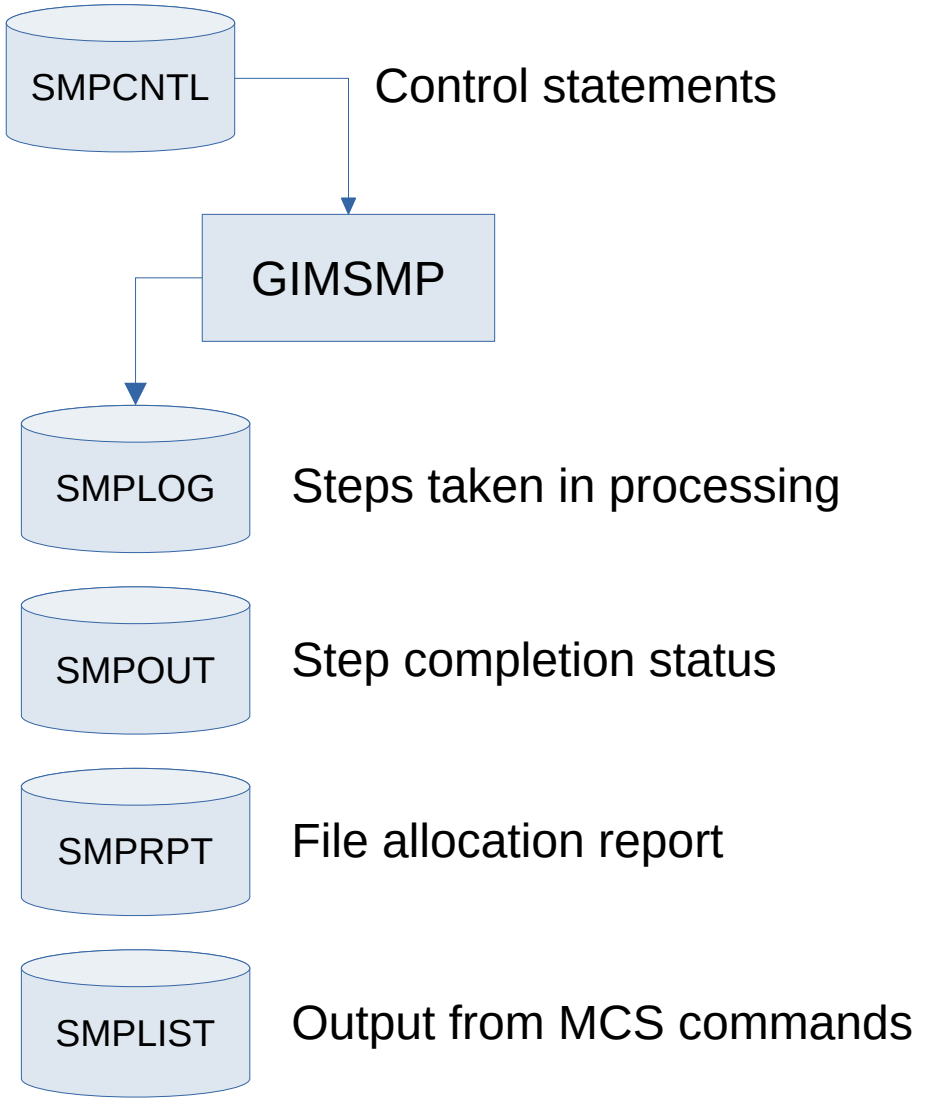
SMP/E is a batch utility program. When people talk about SMP/E "commands" they're referring to the source statements in a declarative language called MCS (Modification Control Statements), and not to individual commands entered interactively.

The name of the program is GIMSMP – GIM is the IBM product prefix for SMP/E. GIMSMP reads the control statements (or "commands") from a data set with DD name SMPCNTL.

SMPCNTL contains 80-byte card-image records. As with control statements for other IBM utilities, you can code the commands anywhere between columns 1 and 72.

```
//SMPCNTL  DD *
 SET BOUNDARY(DLB31).
 LIST CLIST(ACBQADG4).
/*
```

```
DDNAME   StepName ProcStep DSID Owner
JESMSGLG JES2                2 IBMUSER
JESJCL   JES2                3 IBMUSER
JESYSMSG JES2                4 IBMUSER
SMPLOG   VERIFY            102 IBMUSER
SMPOUT   VERIFY            103 IBMUSER
SMPRPT   VERIFY            104 IBMUSER
SMPLIST  VERIFY            105 IBMUSER
```



RUNNING SMP/E IN BATCH

```
                          CSI QUERY - CLIST    ENTRY          Row 1 to 1 of 1
===> _                                                       SCROLL ===> CSR

  To return to previous panel, enter END .

  Primary Command: FIND

  Entry Type:  CLIST                                    Zone
  Entry Name:  ACBQADG4                                 Zone

                                          LASTUPD: HDZ3310

FMID      HDZ3310
RMID      HDZ3310
DISTLIB   ADGTCLIB
SYSLIB    DGTCLIB
          -------- -------- -------- -------- -------- ------

 ****************************** Bottom of data ************
```

SMP/E ISPF query output

Same query in batch, contents of SMPLIST.

```
 SDSF OUTPUT DISPLAY IBMUSERZ JOB01456  DSID   105 LINE 0       COLUMNS 02- 81
 COMMAND INPUT ===> _                                        SCROLL ===> CSR
********************************* TOP OF DATA ***********************************
PAGE 0001  - NOW SET TO DLIB   ZONE DLB31    DATE 06/24/26  TIME 06:10:05  SMP/E

 LIST CLIST(ACBQADG4).
PAGE 0002  - NOW SET TO DLIB   ZONE DLB31    DATE 06/24/26  TIME 06:10:05  SMP/E

DLB31   CLIST    ENTRIES


  NAME

ACBQADG4  LASTUPD          = HDZ3310  TYPE=ADD
          LIBRARIES        = DISTLIB=ADGTCLIB  SYSLIB=DGTCLIB
          FMID             = HDZ3310
          RMID             = HDZ3310

PAGE 0003  - NOW SET TO DLIB   ZONE DLB31    DATE 06/24/26  TIME 06:10:05  SMP/E

LIST     SUMMARY REPORT FOR DLB31

ENTRY-TYPE    ENTRY-NAME         STATUS

CLIST         ACBQADG4           STARTS ON PAGE 0002
******************************** BOTTOM OF DATA *********************************
```

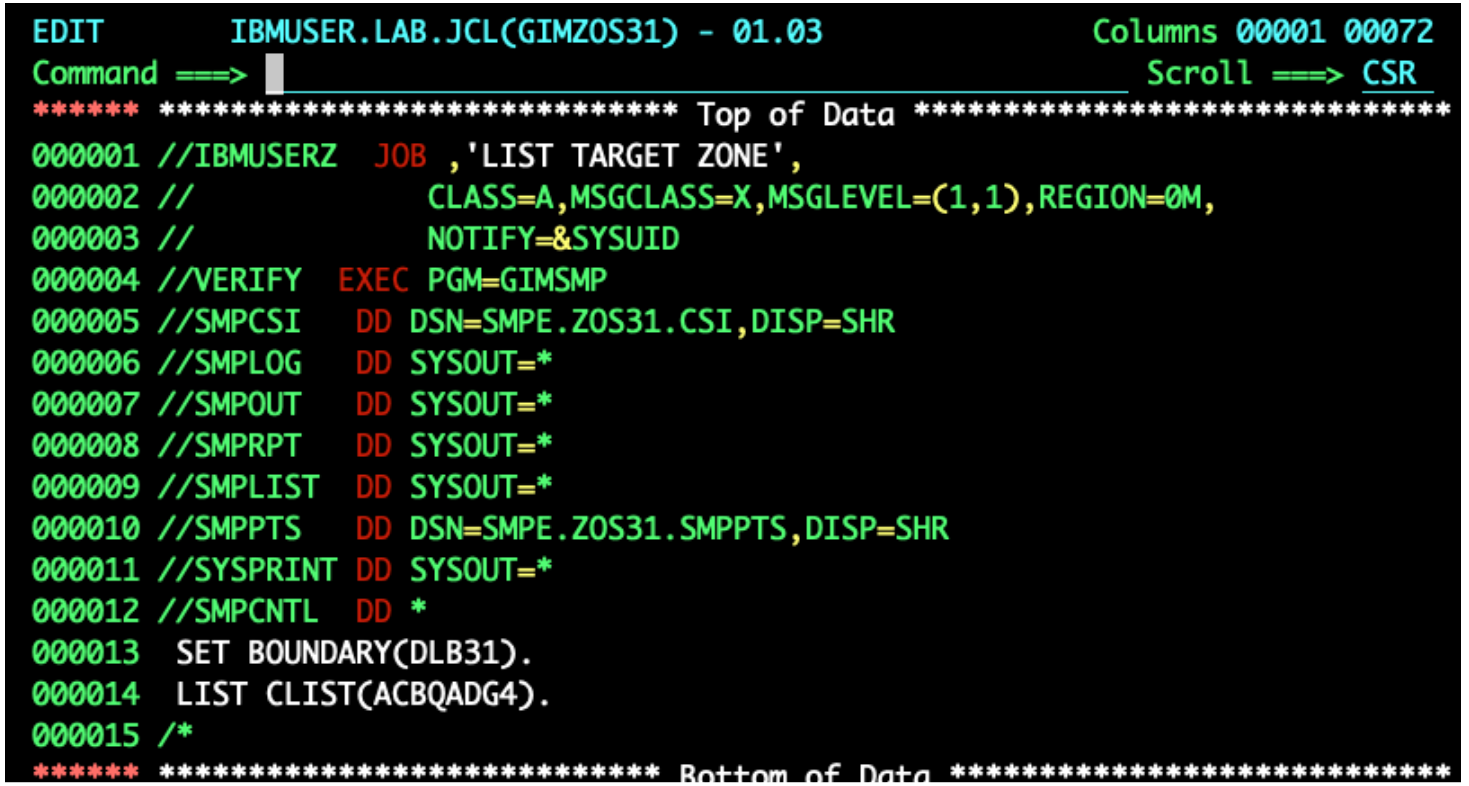
RUNNING SMP/E IN BATCH

This is the JCL that produced the output shown on the preceding slides.

DDNAME SMPCSI identifies the CSI we're operating against.

You can see the DD statements for the output data sets we saw in the SDSF screenshot.

The MCS narrows the query down to zone DLB31, Entry Type "CLIST", and Entry "ACBQADG4."



For queries in batch, you want to narrow the search as much as you can, because the job may run a long time and generate huge amounts of spool data if the query is too broad.

Rule of thumb: Use ISPF for queries when you aren't sure what's in a CSI. Run GIMSMP in batch when you're ready to apply system modifications.

NEXT STEPS

There are two things people do with SMP/E.

System programmers who work at companies that are IBM customers use it to install and update software packages from IBM and independent software vendors (ISVs).

IBM and ISV's use SMP/E to prepare packages according to IBM's standards so that customers can install their products using SMP/E.

NEXT STEPS

In the next few labs, we want to get our hands on SMP/E without getting overwhelmed by the complexity of real SYSMODs and the tool's arcane commands and workflows.

We need a small, simple product that provides a real-world(ish) use case for SMP/E, so we can see what the various pieces are and how they relate to each other.

To that end, we will:

- Prepare a trivial software product for distribution as a SYSMOD
- Install the SYSMOD on z/OS

CREATING A TRIVIAL PRODUCT TO INSTALL

Real products have an arbitrary number of files and often have a complicated requisite chain and numerous dependencies. It's hard to learn about something as arcane and user-unfriendly as SMP/E by plunging straight into the deep end with a cinder block tied to your ankle.

Let's make up a fake product that has only a few moving parts, no dependencies, and no special build or installation requirements that would muddy the waters.

It should be something that looks more-or-less useful, but isn't really. It should also offer a bit of fun. Something like a *chindogu* ("curious device"). A chindogu must be almost useful, a.k.a. unuseless.

I'm sure you've seen – and used – software like that.

Butter Stick

Cat Mop

Swiss Army Gloves

CREATING A TRIVIAL PRODUCT TO INSTALL

Our product is a collection of almost-useful utilities, including:

- GREET "Hello, World!" in multiple languages
- SETCC Like IEFBR14, but sets a specified condition code

Inspired by the Japanese fad of the 1990s, we'll call it the *Chindogu Utility Mega-Pack*, or CHUMP.

We're going to create some libraries such as a software development team would use, and define several artifacts that are used to make and use CHUMP.

Then we'll prepare a SYSMOD suitable for installation on z/OS using SMP/E. Finally we'll use another SMP/E setup to install it.

We'll be practicing the sequence of steps and the command language of SMP/E using a realistic project structure for an unrealistic product.

Hay Fever Headset

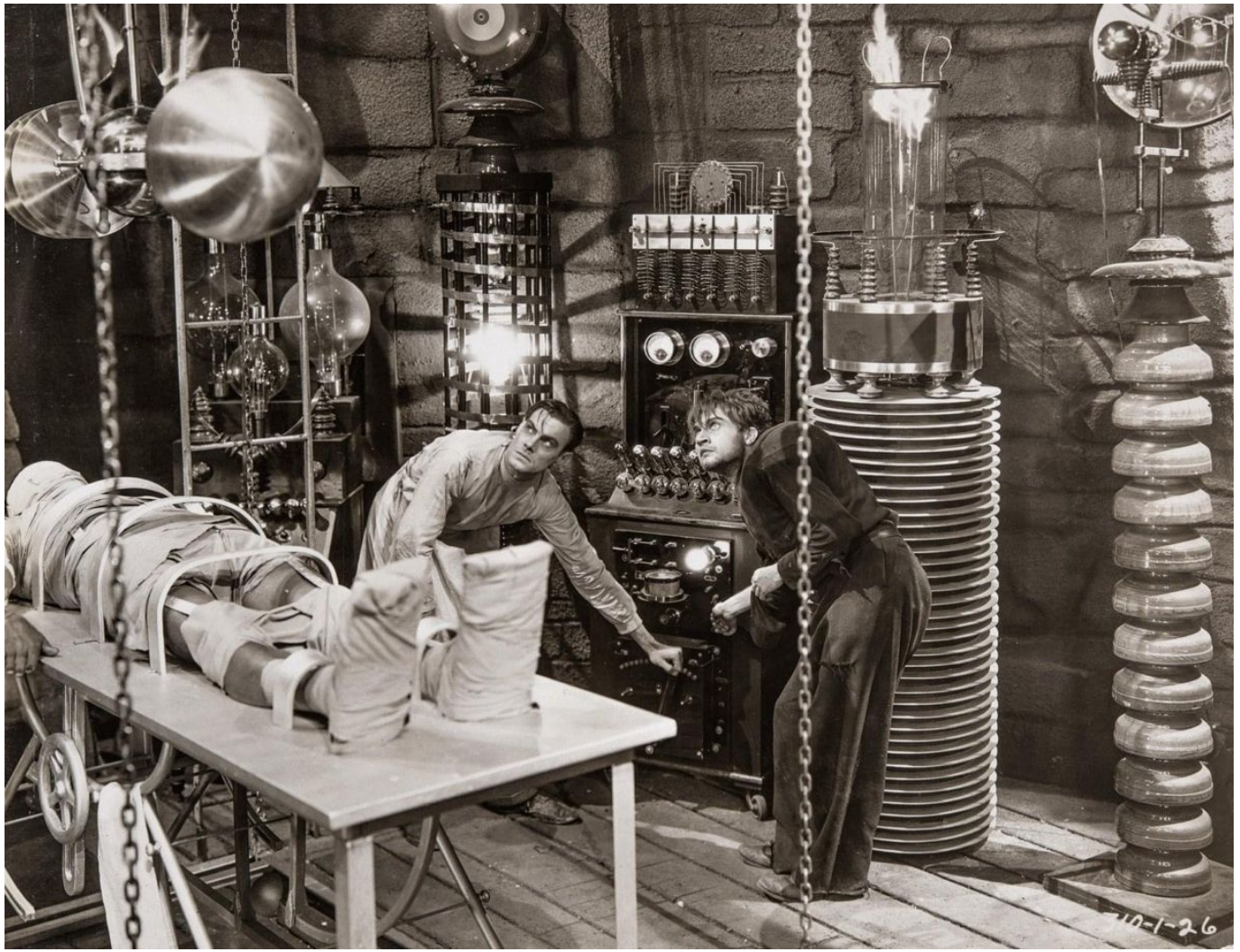
CHUMP DEVELOPMENT LIBRARIES & MEMBERS

Umbrella Tie

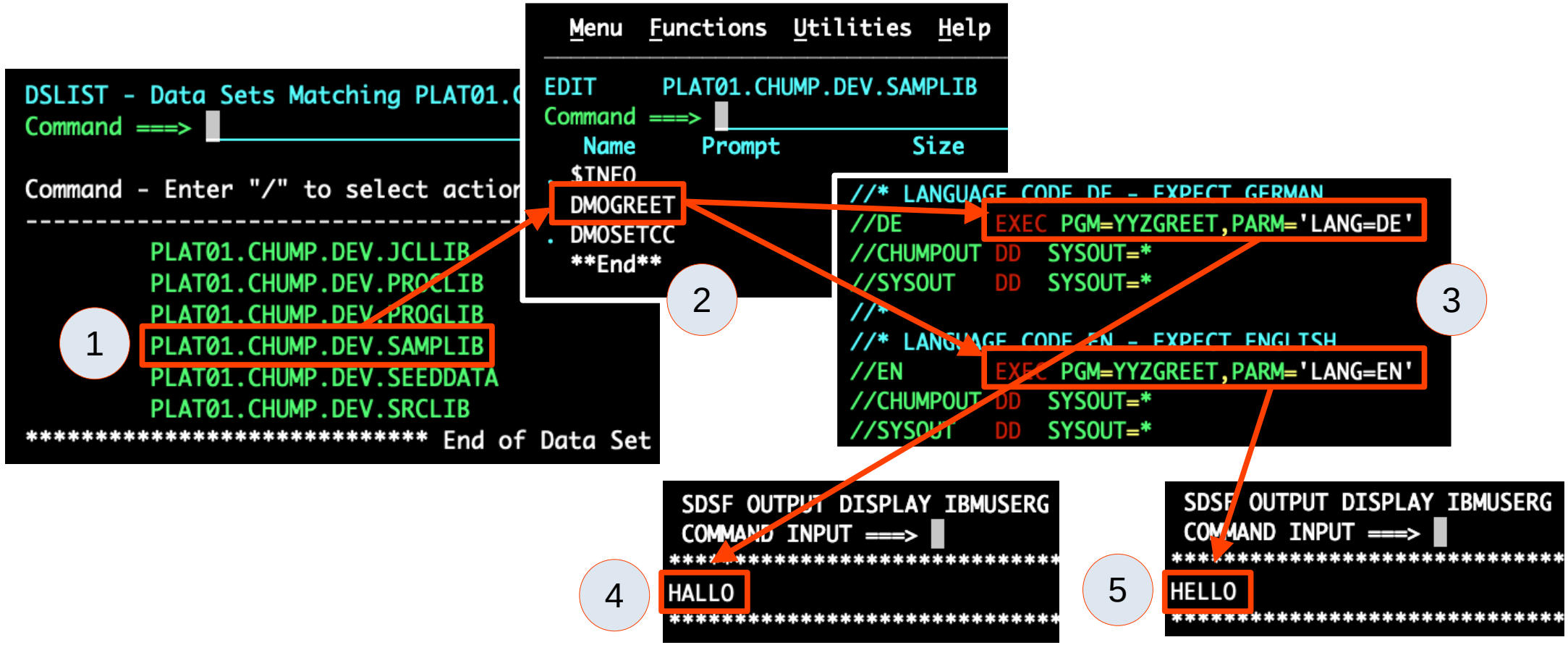
The Chindogu Utility Mega-Pack development environment consists of the following libraries and members:

<userid>.CHUMP.DEV.SRCLIB	Source code library
• YYZGREET	Source for YYZGREET
• YYZSETCC	Source for YYZSETCC
<userid>.CHUMP.DEV.JCLLIB	JCL library
• YYZBDJCL	Job to build all utilities
<userid>.CHUMP.DEV.PROCLIB	JCL Procedures library
• YYZBUILD	PROC to build one utility
<userid>.CHUMP.DEV.SAMPLIB	Samples library
• $INFO	Product documentation
• DMOGREET	Demo job for YYZGREET
• DMOSETCC	Demo job for YYZSETCC
<userid>.CHUMP.DEV.PROGLIB	Executables

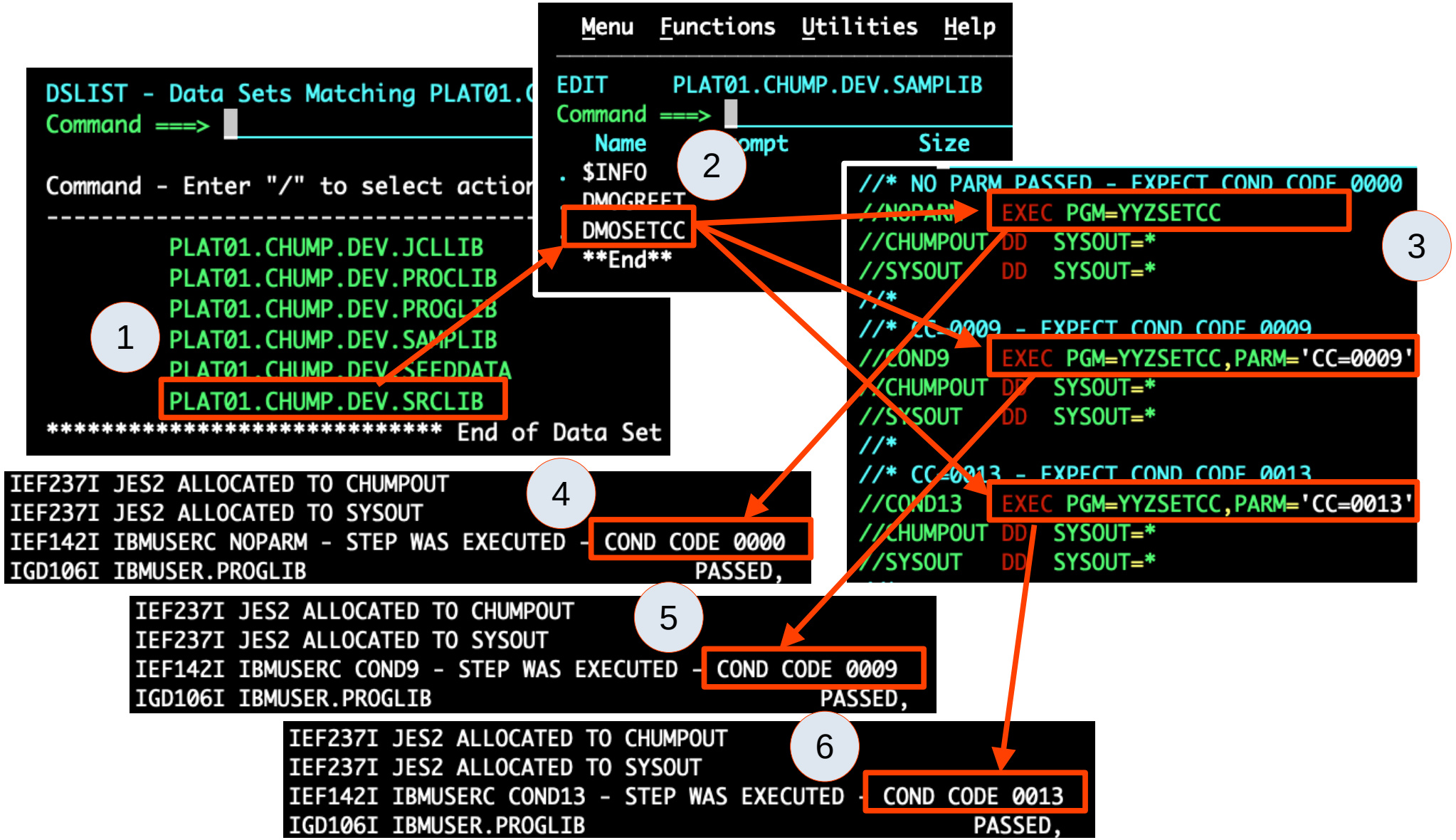
LAB Z/OS 38: BUILD & RUN THE CHUMP UTILITIES



RUNNING THE CHUMP UTILITIES



RUNNING THE CHUMP UTILITIES



REWORK AT THIS POINT

reworking this part of the presentation

SANDBOX FOR PLAYING WITH SMP/E

We should be able to set up a sandbox area where we can practice SMP/E work without any risk to the system.

What will we need?

- Define and initialize one or more CSI data sets
- Allocate SMP/E working data sets PTS, LOG, etc.
- Define the Zones in the CSI data set
- Allocate DDDEF data sets and define them to SMP/E
- Create a trivial FMID and SYSMOD

SMP/E SANDBOX DATA SET NAMING CONVENTION

名可名

The name that can be named

We can use our userids as HLQs for our sandbox SMP/E data sets.

`<userid>.CHUMP.CSI`

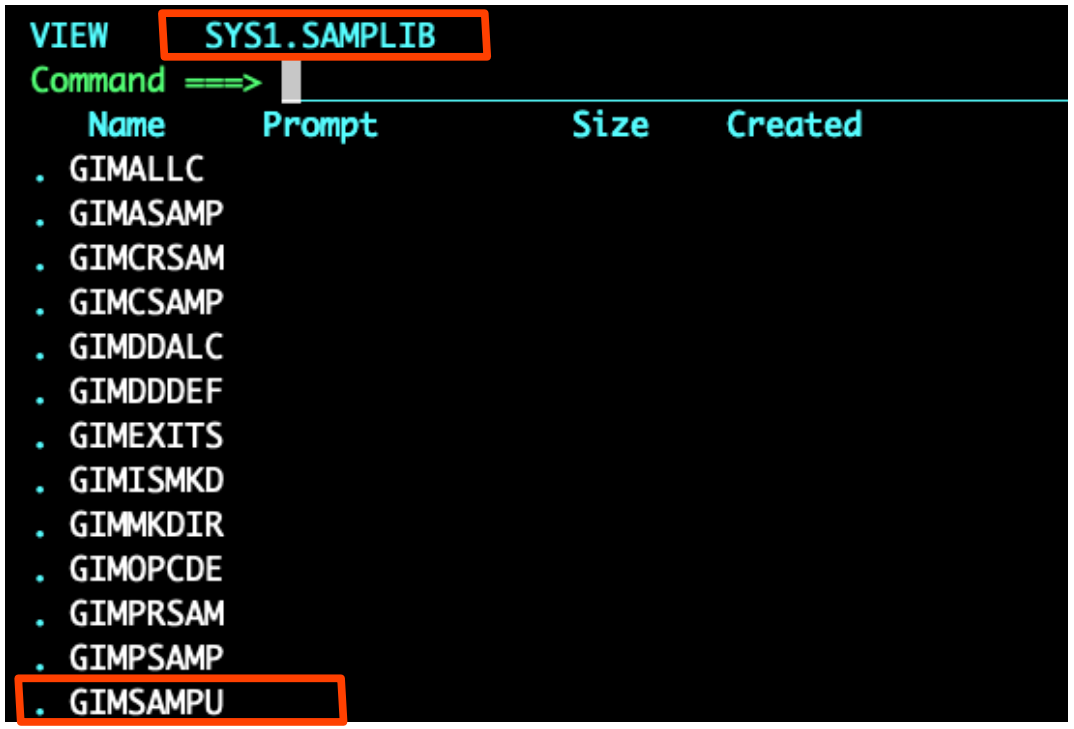
`<userid>.CHUMP.SMPPTS`

`<userid>.CHUMP.SMPLOG`

etc.

Why "chump?" It's the name of the fake product we'll install.

THE SYS1.SAMPLIB DATA SET



Many of the products IBM and others ship for the z/OS environment are pretty complicated to install, configure, and use. SMP/E itself is like that.

Setting up a sandbox SMP/E environment from scratch would be a non-trivial exercise.

Fortunately, we have the SYS1.SAMPLIB data set to help us.

Most of the components that ship with z/OS and most of the products we might install provide members for the SYS1.SAMPLIB library.

Members whose names begin with GIM, the product prefix for SMP/E, contain examples for defining, configuring, and using SMP/E.

A day in the life of a z/OS system programmer often includes building jobs based on members in the sample library. The sample members are usually not runnable without some tweaking, and many of them contain only bits and pieces of jobstreams, configuration settings, and other items. They also contain copious comments, instructions, and information about dependencies and gotchas.

We'll use one of these members, GIMSAMPU, to help us set up our sandbox SMP/E data sets.

USING JCL SYMBOLS IN A SYSIN DATA SET

Depending on how you choose to set up your SMP/E sandbox data sets, you may need to run jobs from your userid but specify a different high-level qualifier for the data sets.

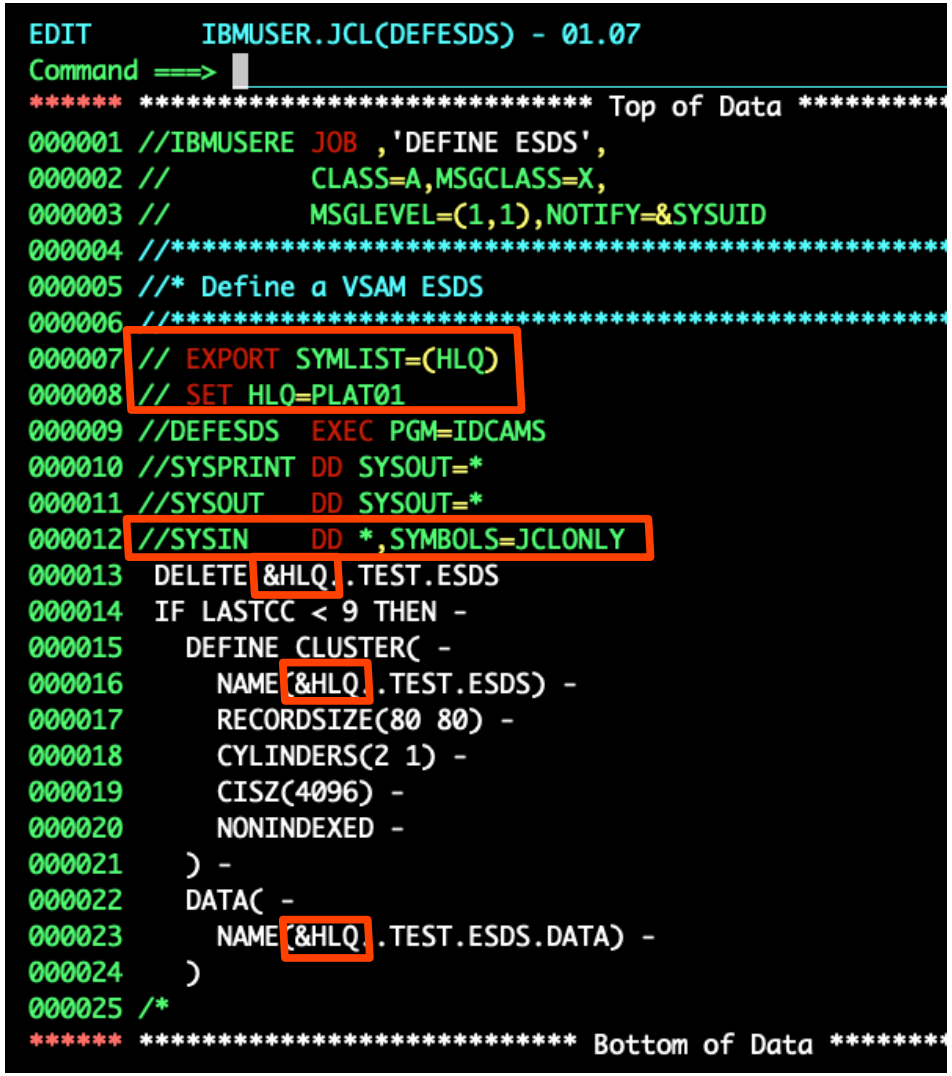
Rather than hard-coding the data set names, you can use symbol substitution in inline SYSIN data sets. Inline data is not JCL, so we have to tell JES2 about the symbols.

1. EXPORT statement (before SET statements)
2. SET statment(s) (before job step where referenced)
3. SYMBOLS= parameter on DD statement

- SYMBOLS=JCLONLY will convert only symbols defined in the JCL (like &HLQ in this example).
- SYMBOLS=EXECSYS will convert system symbols (like &SYSUID) as well as those defined in the JCL.
- SYMBOLS=CNVTSYS causes JES2 to substitute symbol values during JCL conversion rather than execution.

Application developers and other general users can't create data sets with high-level qualifiers other than their own userids, but as a Platform Engineer you will often need to do so.

The example at right is general, and not part of SMP/E setup.



SYS1.SAMPLIB(GIMSAMPU)

Member GIMSAMPU in SYS1.SAMPLIB contains an incomplete jobstream with three steps. The steps create and initialize quite a few data sets that SMP/E uses.

Like many members in SYS1.SAMPLIB, this one begins with a long block of comments that state the purpose of the job and provide guidance in filling in or modifying the portions that need to be finalized before running it.

```
VIEW       SYS1.SAMPLIB(GIMSAMPU) - 01.00                Columns 00001 00072
Command ===>                                               Scroll ===> CSR
****** ***************************** Top of Data ******************************
000001 //GIMSAMPU JOB job parameters
000002 //*
000003 //* Allocate and prime SMP/E CSI and operational data sets.
000004 //*
000005 //*********************************************************************
000006 //*    Licensed Materials - Property of IBM
000007 //*    5650-ZOS
000008 //*    Copyright IBM Corp. 1982, 2019
000009 //*********************************************************************
000010 //*
000011 //* This JCL sample can be used to allocate SMP/E CSI data sets, as
000012 //* well as other SMP/E operational data sets.  This sample can also
000013 //* be used to prime the CSI data sets with basic SMP/E zone
000014 //* definitions, OPTIONS entries, and DDDEF entries.
000015 //*
000016 //* The space values for all data sets are approximate and may need
000017 //* to be adjusted based on your specific needs.
000018 //*
000019 //* CAUTION: This is neither a JCL procedure nor a complete JCL job.
000020 //* It is a sample.  Before using this sample, you will have to make
000021 //* the following modifications:
```

SYS1.SAMPLIB(GIMSAMPU)

The first step in the job, DEFZONES, uses IDCAMS to define CSI data sets and initialize them with a dummy record.

You may recall that VSAM data sets are in an "empty" state immediately after they're defined, and that doesn't simply mean they contain no logical records; it means they've never contained any records.

In that state, they can't be opened for input. The SMP/E batch program, GIMSMP, has to open CSI data sets for input and output.

This step takes care of that by writing data from GIMZPOOL, specifically designed to initialize a CSI properly. Doing that *correctly* by hand, from scratch, would be tedious, frustrating, and time-consuming.

```
VIEW       SYS1.SAMPLIB(GIMSAMPU) - 01.00                Columns 00001 00072
Command ===>                                               Scroll ===> CSR
000071 //*******************************************************************
000072 //*
000073 //* Allocate VSAM data sets to contain SMP/E CSIs, and initialize
000074 //* them by copying the GIMZPOOL seed record.
000075 //*
000076 //*******************************************************************
000077 //*
000078 //DEFZONES EXEC PGM=IDCAMS
000079 //SYSPRINT DD SYSOUT=*
000080 //GIMZPOOL DD DSN=SYS1.MACLIB(GIMZPOOL),DISP=SHR
000081 //SYSIN    DD *
000082   DEFINE CLUSTER(                          +
000083            NAME(&HLQ&.GLOBAL.CSI)          +
000084            VOLUMES(&VOLUME&)               +
000085            CYLINDERS(100 10)               +
000086            FREESPACE(10 5)                 +
000087            KEYS(24 0)                      +
000088            RECORDSIZE(24 143)              +
000089            SHAREOPTIONS(2 3)               +
000090                )                           +
```

SYS1.SAMPLIB(GIMSAMPU)

Something else to notice is that the sample JCL contains various placeholders that we have to change before we can run the job.

```
VIEW       SYS1.SAMPLIB(GIMSAMPU) - 01.00                Columns 00001 00072
Command ===>                                               Scroll ===> CSR
000071 //*******************************************************************
000072 //*
000073 //* Allocate VSAM data sets to contain SMP/E CSIs, and initialize
000074 //* them by copying the GIMZPOOL seed record.
000075 //*
000076 //*******************************************************************
000077 //*
000078 //DEFZONES EXEC PGM=IDCAMS
000079 //SYSPRINT DD SYSOUT=*
000080 //GIMZPOOL DD DSN=SYS1.MACLIB(GIMZPOOL),DISP=SHR
000081 //SYSIN    DD *
000082   DEFINE CLUSTER(                          +
000083          NAME(&HLQ&.GLOBAL.CSI)            +
               VOLUMES(&VOLUME&)                 +
               CYLINDERS(100 10)                 +
               FREESPACE(10 5)                   +
               KEYS(24 0)                        +
               RECORDSIZE(24 143)                +
               SHAREOPTIONS(2 3)                 +
000090             )                             +
```


```
LUSTER(
  NAME(&HLQ&.GLOBAL.CSI)
  VOLUMES(&VOLUME&)
```

SYS1.SAMPLIB(GIMSAMPU)

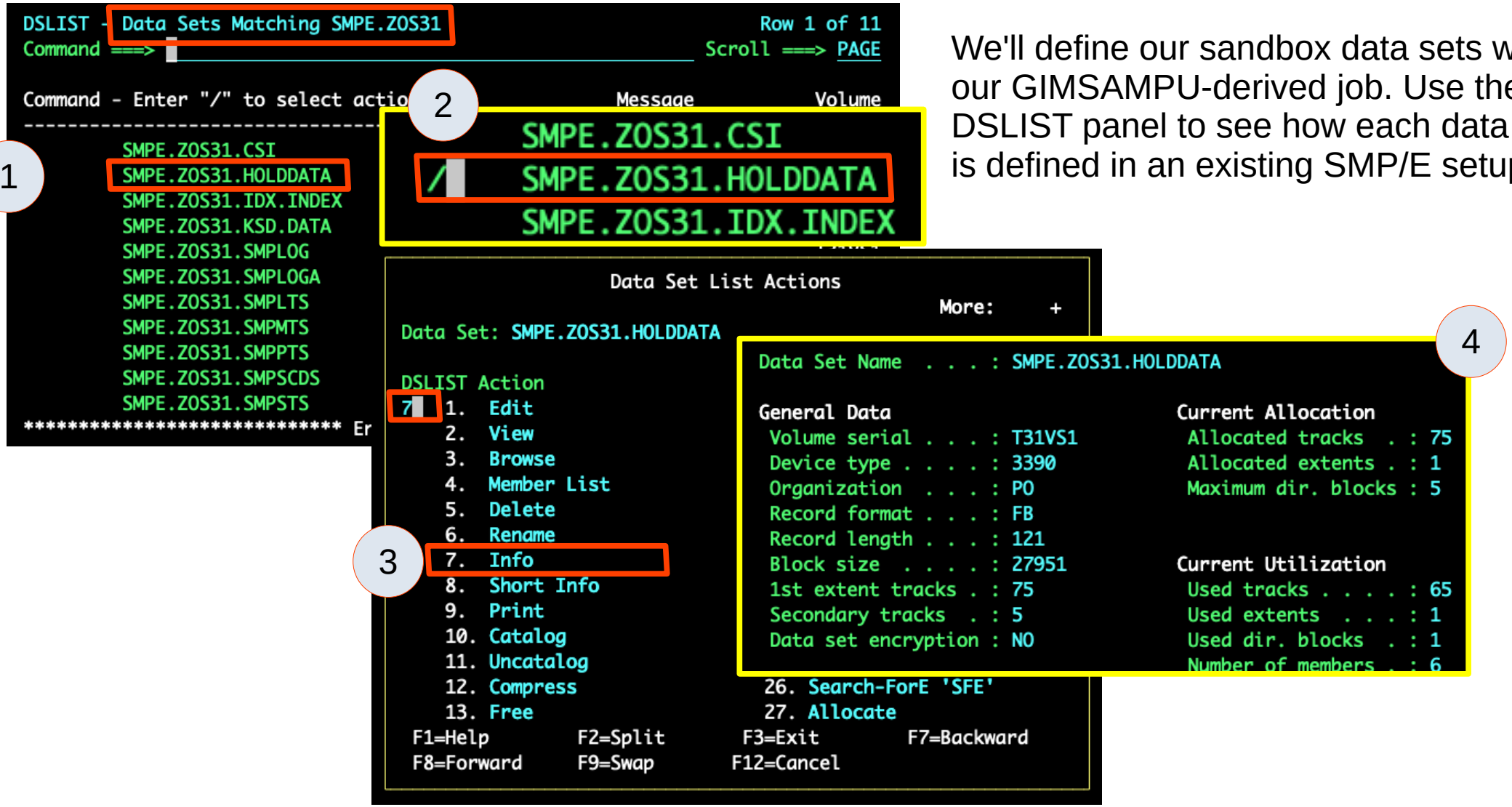
The second step, ALLOCDS, uses IEFBR14 to allocate the PTS, MTS, STS, and SCDS as well as various log data sets.

```
//ALLOCDS  EXEC  PGM=IEFBR14,COND=(0,LT)
//*
//*  Recommend SMPPTS be a PDSE (DSNTYPE=LIBRARY)
//*
//SMPPTS   DD DSN=&HLQ&.SMPPTS,
//         DISP=(NEW,CATLG),
//         DCB=(RECFM=FB,LRECL=80,BLKSIZE=0,DSORG=PO),
//         DSNTYPE=LIBRARY,
//         UNIT=&UNIT&,
//         VOL=SER=&VOLUME&,
//         SPACE=(CYL,(250,50,50))
```

The third step, UPDZONES, uses SMP/E's own functionality to finish defining the Zone information in the CSI and to add DDDEFs that point SMP/E at various work data sets so that we needn't code DD statements for them all.

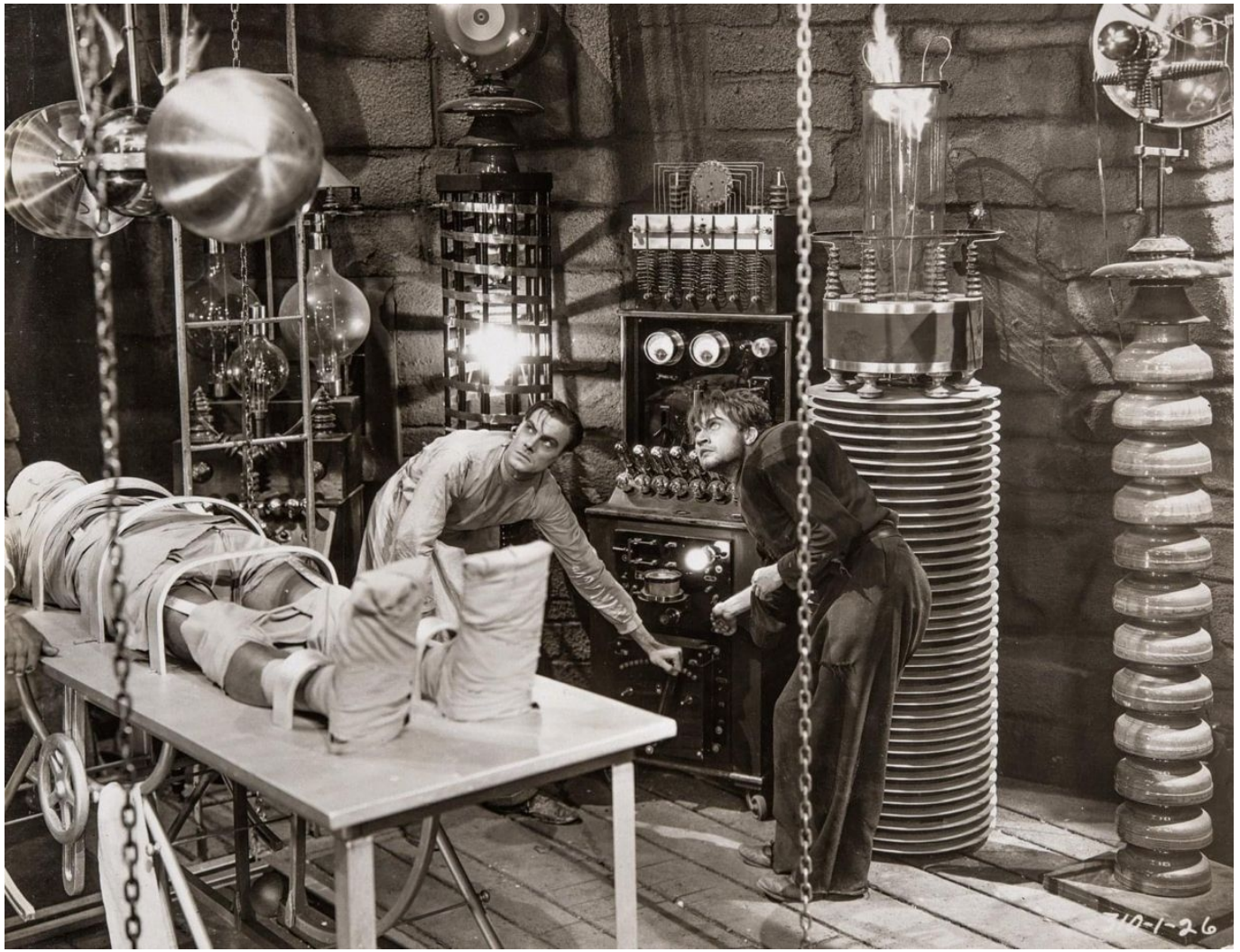
```
//SMPCNTL  DD *
  SET BOUNDARY(GLOBAL).
  UCLIN.
    ADD OPTIONS(GOPT)
          DSPREFIX(&HLQ&.SMPTLIB)
          DSSPACE(20,20,100)
          MSGFILTER(YES)
          MSGWIDTH(80)
          RECZGRP(ALLZONES)
          RETRYDDN(ALL).
```

OTHER KEY SMP/E SANDBOX DATA SETS



We'll define our sandbox data sets with our GIMSAMPU-derived job. Use the DSLIST panel to see how each data set is defined in an existing SMP/E setup.

LAB Z/OS 38: DEFINE SMP/E DATA SETS FOR PACKAGING YOUR PRODUCT



CHECKING THE RESULTS OF LAB 38

You can use JCL like this to see what your SMP/E CSI contains immediately after you initialize it with the bare minimum information.

The SET BOUNDARY commands tell GIMSMP which Zone to look at. BOUNDARY is usually abbreviated as BDY. The LIST command tells GIMSMP to list all the entries in the Zone.

In this example, the userid is PLAT01.

The HLQ symbol includes the userid and the product name, CHUMP, because this CSI will be used to prepare the product for distribution.

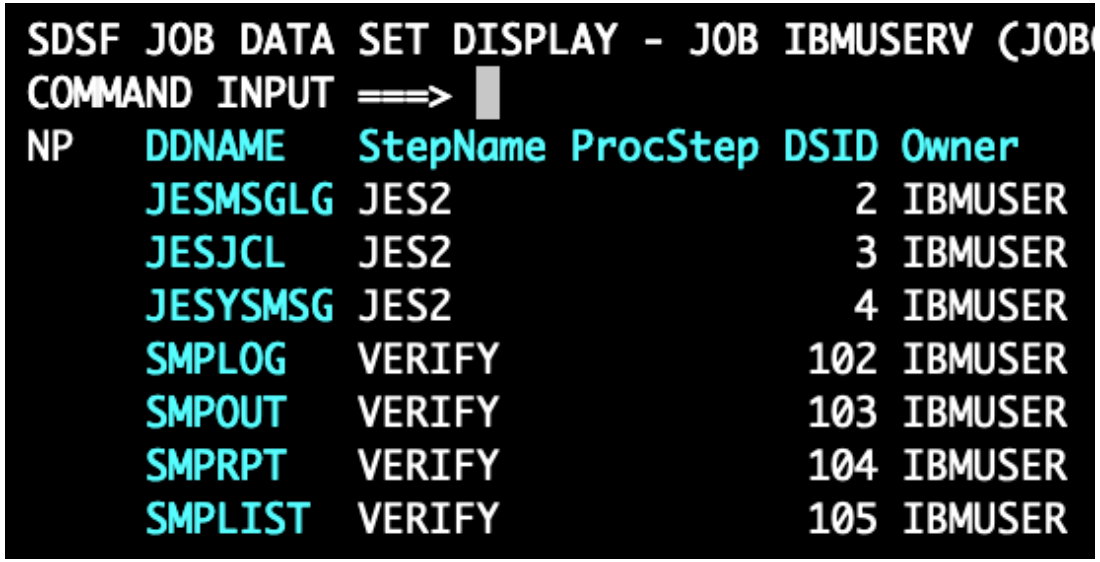
On the receiving side, the CSI name won't include a product name, since it will be used to manage many products.

```
//IBMUSERV  JOB ,'VERIFY SANDBOX CSI',
//             CLASS=A,MSGCLASS=X,MSGLEVEL=(1,1),
//             REGION=0M,
//             NOTIFY=&SYSUID
// EXPORT SYMLIST=(HLQ)
// SET HLQ=PLAT01.CHUMP
//VERIFY  EXEC PGM=GIMSMP
//SMPCSI   DD DSN=&HLQ..GLOBAL.CSI,DISP=SHR
//SMPLOG   DD SYSOUT=*
//SMPOUT   DD SYSOUT=*
//SMPRPT   DD SYSOUT=*
//SMPLIST  DD SYSOUT=*
//SMPPTS   DD DSN=&HLQ..SMPPTS,DISP=SHR
//SYSPRINT DD SYSOUT=*
//SMPCNTL  DD *,SYMBOLS=JCLONLY
 SET BDY(GLOBAL).   /* List the contents of the Global zone */
 LIST.
 SET BDY(TARGET).   /* ...and the Target zone */
 LIST.
 SET BDY(DLIB).     /* ...and the DLIB zone */
 LIST.
/*
```

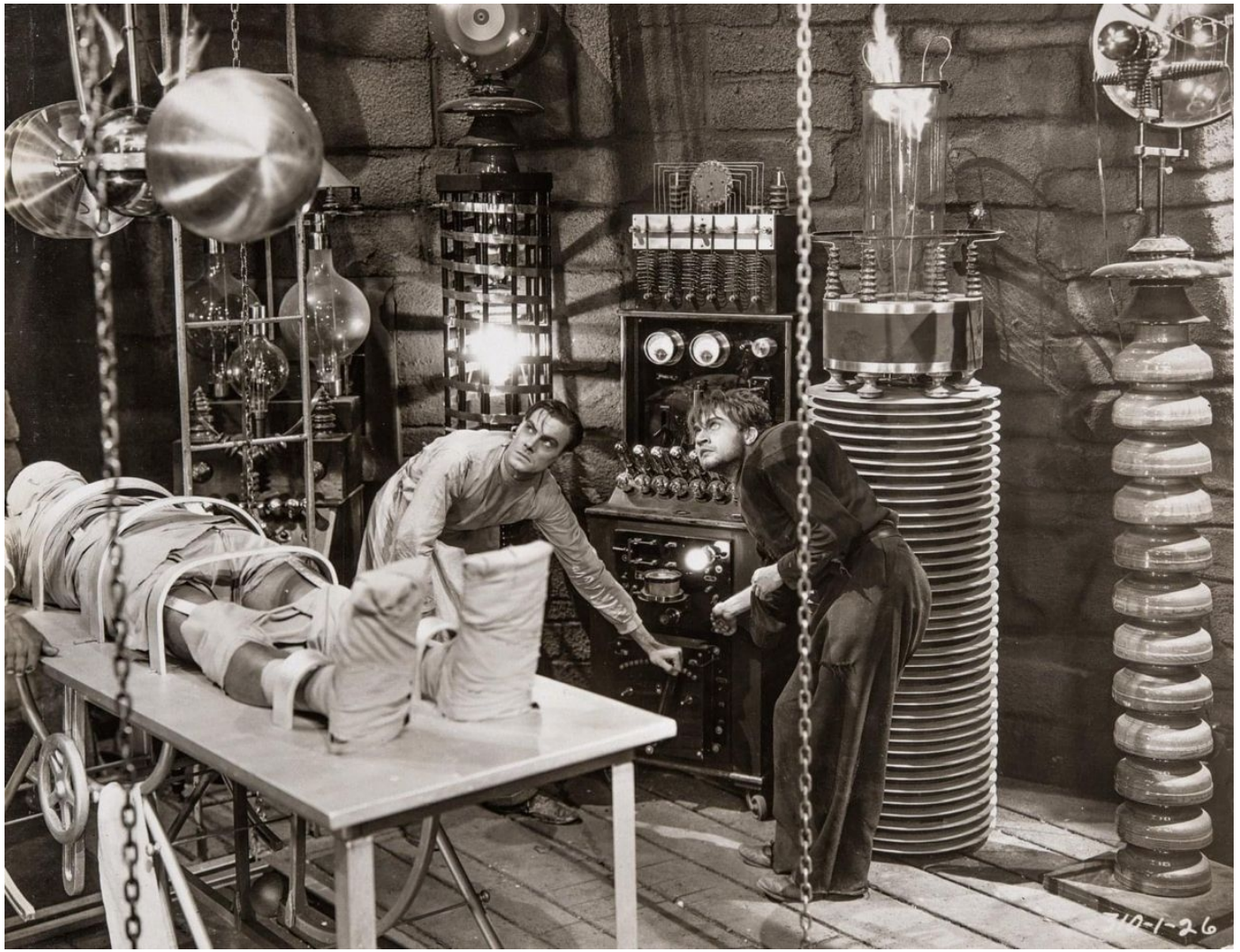
CHECKING THE RESULTS OF LAB 38

These are the data sets produced by the list job. They were routed to SYSOUT, so they appear as spooler data sets under SDSF.

Read through them and familiarize yourself with the kinds of information each one provides. Also look for the logical links between the different zones, and how SMP/E defines several work data sets that it uses during processing.



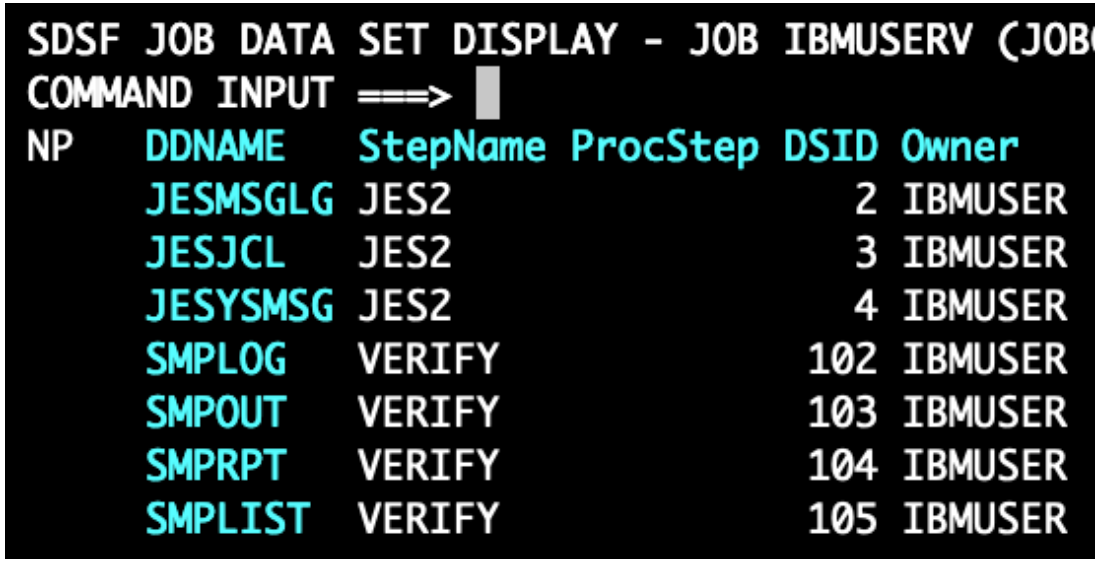
LAB Z/OS 39: ADD DDDEFS TO THE CSI



CHECKING THE RESULTS OF LAB 39

These are the data sets produced by the list job. They were routed to SYSOUT, so they appear as spooler data sets under SDSF.

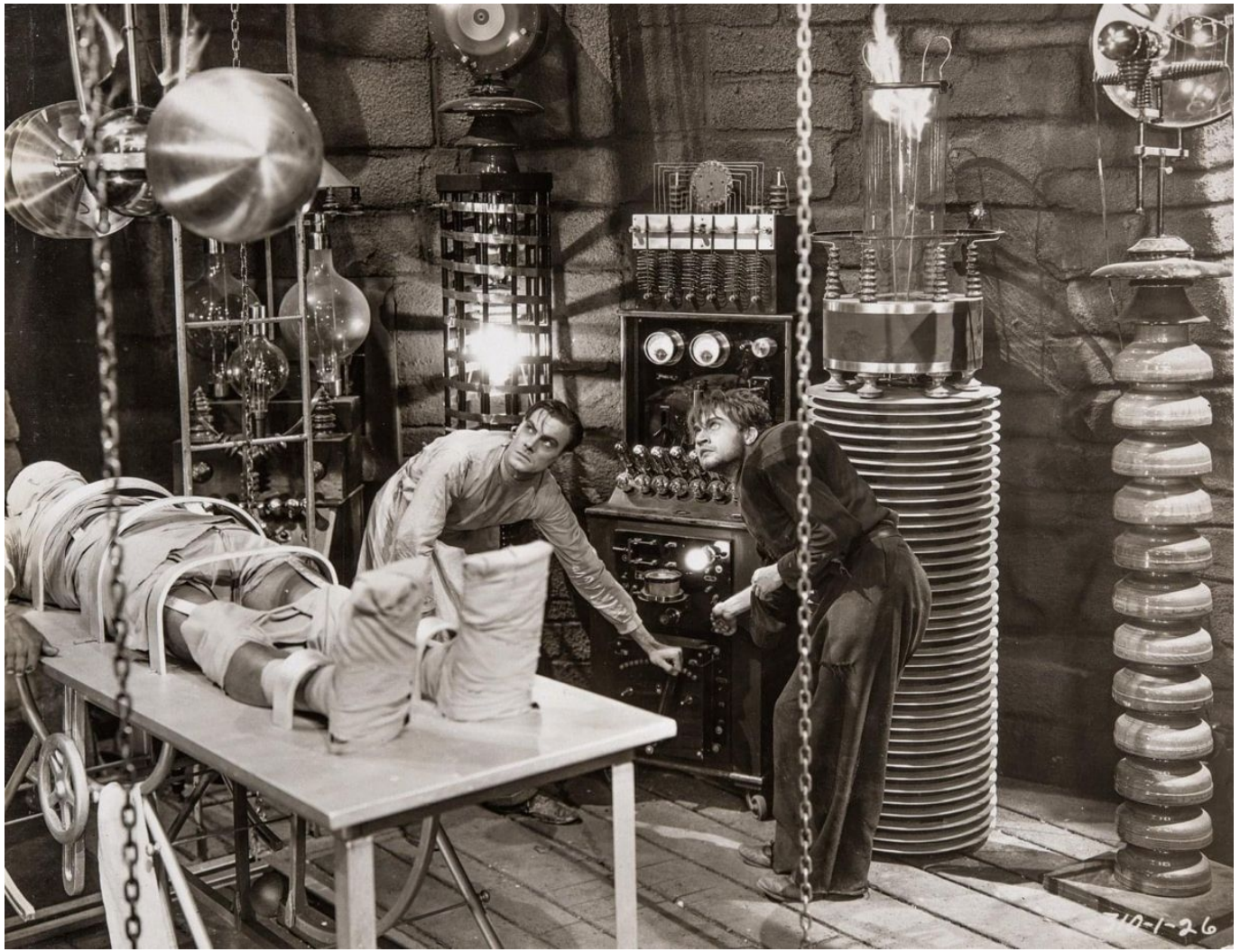
Read through them and familiarize yourself with the kinds of information each one provides. Also look for the logical links between the different zones, and how SMP/E defines several work data sets that it uses during processing.



SMP/E ELEMENT TYPES

Program Object	PROGRAM
Load Module	MOD
Source (Assembler, COBOL, JCL, etc.)	SRC
Macro	MAC
JCL Procedure	PROC
CLIST	CLIST
REXX exec	EXEC
ISPF panel	PNL
ISPF skeleton	SKL
ISPF message member	MSG
ISPF help/tutorial text	HLP
General text member	TEXT

LAB Z/OS 40: ADD ELEMENTS TO THE CSI

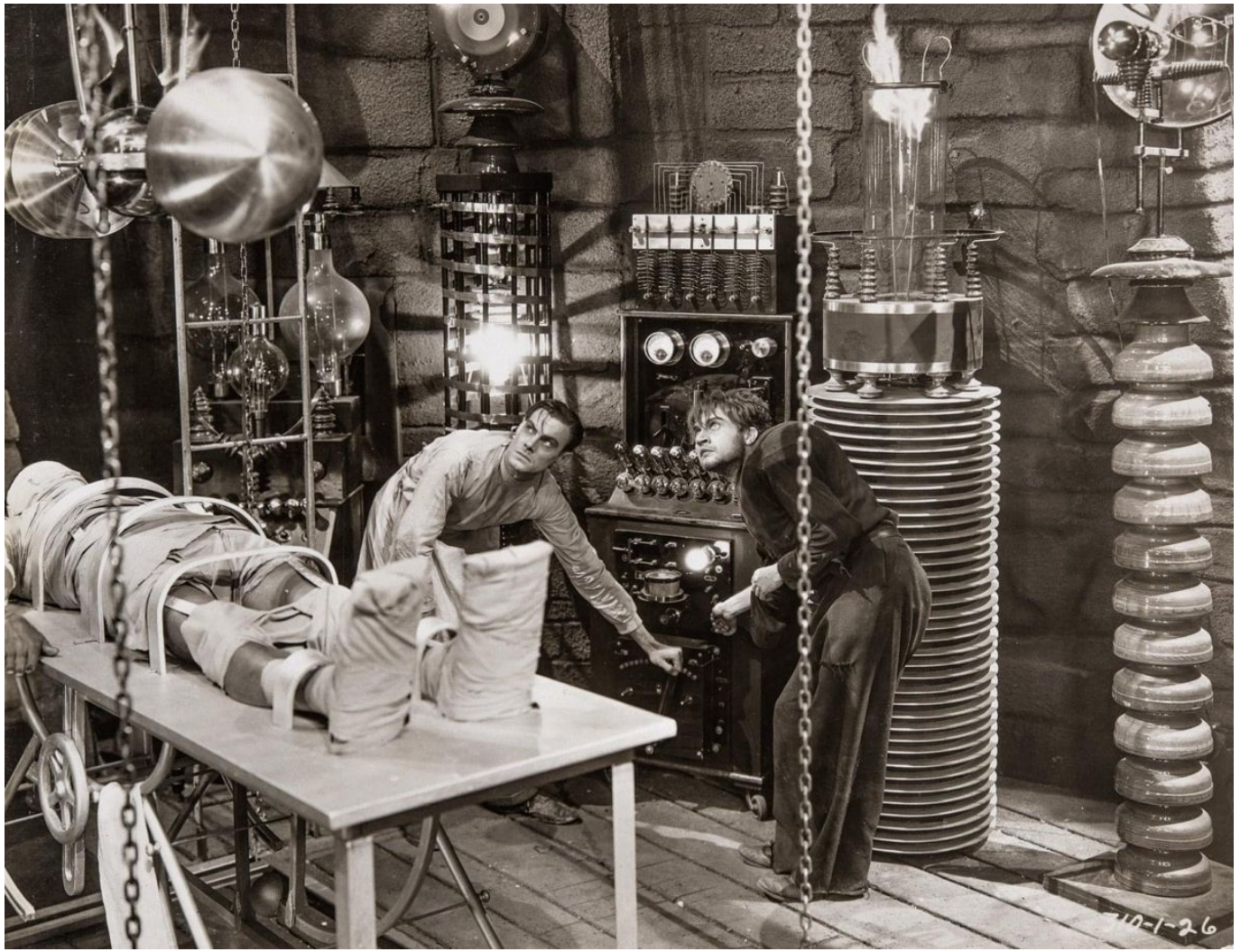


CHECKING THE RESULTS OF LAB 39

The results of LIST commands for the elements you defined should show that the elements are in the CSI in the appropriate zones.

```
SDSF JOB DATA SET DISPLAY - JOB IBMUSERV (JOB
COMMAND INPUT ===>
NP    DDNAME   StepName ProcStep DSID Owner
      JESMSGLG JES2                 2 IBMUSER
      JESJCL   JES2                 3 IBMUSER
      JESYSMSG JES2                 4 IBMUSER
      SMPLOG   VERIFY             102 IBMUSER
      SMPOUT   VERIFY             103 IBMUSER
      SMPRPT   VERIFY             104 IBMUSER
      SMPLIST  VERIFY             105 IBMUSER
```

LAB Z/OS 41: CREATE SMP/E PACKAGE



WHAT YOU KNOW AND WHAT YOU CAN DO

- You know the role of SMP/E in z/OS system management.
- You can locate and browse information in SMP/E data sets using ISPF panels.
- You are somewhat familiar with the kinds of information stored in each SMP/E data set and how SMP/E uses the information.
-

WHAT'S NEXT

- Software Installation and Management